Design, testing and development of a water-rocket glider

Report of the work done for the competition
Airbus Sloshing Rocket Workshop

AIRBUS

Authors:
Piercarlo Mirto, Vincenzo Junior Di Rosa,
Gabriele Salomone, Maria Giulia Monti,
Pierluca De Felice, Maria Mattiello.

Contents

I The aircraft model 13

1 Concept design 15
1.1 Flight path . . . 15
1.1.1 Rocket mode . . . 16
1.1.2 Glider mode . . . 17
1.1.3 Mission programming . . . 17

2 Detailed design 19
2.1 Aircraft Design . . . 19
2.1.1 Main design and adopted solutions . . . 19
2.1.2 Possible applications . . . 22
2.2 Iterative cycle . . . 24
2.2.1 `Matlab` code . . . 24
2.2.2 Mindset . . . 24
2.2.3 Results . . . 26
2.3 Propulsion . . . 27
2.3.1 Launch base . . . 28
2.3.2 Launch procedure . . . 29

3 Control System Implementation 31
3.1 Component description . . . 31
3.2 IDE . . . 32
3.3 Libraries . . . 32
3.4 Program structure and basic idea . . . 34
3.4.1 `while()` . . . 35
3.4.2 `switch()` . . . 35
3.4.3 Flight correction algorithm . . . 36

4 Prototype Construction 37
4.1 Goals and Management . . . 37
4.2 Construction Details . . . 37
4.2.1 Launch base . . . 37
4.2.2 Fuselage . . . 38
4.2.3 Wings and Horizontal plane . . . 39
4.2.4 Control system . . . 40
4.3 Process of Construction . . . 41
4.4 Bill of Materials . . . 42

II Sloshing **45**

5 Introduction to sloshing in the design **47**
5.1 Workflow for sloshing reduction . . . 47
5.1.1 Optimized design features to allow lighter sloshing issue . . . 48
5.1.2 Choice of reference dimensions . . . 48
5.1.3 Choice of reference shape . . . 50
5.1.4 Moving tank simulation and baffles configuration . . . 52
5.2 Final considerations and tank integration . . . 55

III Test & experiments **63**

6 Model validation and Safety assessment **65**
6.1 Model validation . . . 65
6.2 Final Configuration . . . 66
6.3 Safety assessment . . . 67

7 Launch tests **69**
7.1 First day . . . 69
7.2 Second day . . . 70

IV Appendix **75**

A Appendix **77**
A.1 Sloshing considerations . . . 77
A.1.1 Ideas from other fields . . . 77
A.2 Codes . . . 78
A.2.1 Inertial properties . . . 78
A.2.2 The aerodynamics model . . . 79
A.2.3 Complete code . . . 81
A.2.4 Control System . . . 90

List of Figures

1 . . . 10
2 . . . 11

1.1 Expected Trajectory . . . 16

2.1 NACA 6409 Polar graphs . . . 20
2.2 . . . 21
2.3 . . . 22
2.4 Credits: https://www.military.com/equipment/mq-1b-predator . . . 22
2.5 . . . 25
2.6 . . . 26
2.7 Velocity to time graphic . . . 26
2.8 . . . 26
2.9 Bottle Comparison . . . 27
2.10 Thrust to time graphics . . . 28
2.11 Clark's cable tie system . . . 28

3.1 Flowchart of the entire code . . . 34

4.1 Launch base . . . 38
4.2 Fuselage interlocking . . . 38
4.3 Wings sticks . . . 39
4.4 Hotwire device . . . 39
4.5 Tailplane structure . . . 40
4.6 Central Circuit . . . 40
4.7 Electronic system final configuration (without plastic cover) . . . 41

5.1 Schematization of performed task . . . 48
5.2 Natural frequency trend . . . 49
5.3 Hydrodynamic sloshing force along x versus radio for harmonic motion . . . 50
5.4 Hydrodynamic sloshing force along x versus radio for harmonic motion . . . 50
5.5 On the left there is the Moment of the hydrodynamic forces acting on the tank walls about the y-axis, on the right there is the Moment of the hydrodynamic forces acting on the tank bottom about the y-axis versus radio for harmonic motion . . . 51
5.6 On the left there is the Moment of the hydrodynamic forces acting on the tank walls about the y-axis, on the right there is the Moment of the hydrodynamic forces acting on the tank bottom about the y-axis versus radio for harmonic motion . . . 51
5.7 Rectangular tank simulation: instant 1 . . . 52
5.8 Rectangular tank simulation: instant 2 . . . 53
5.9 Rectangular tank simulation: instant 3 . . . 53
5.10 Rectangular tank simulation: instant 4 . . . 54
5.11 Rectangular tank simulation: instant 5 . . . 54

5.12 Rectangular tank simulation: instant 6 . . . 54
5.13 Rectangular tank simulation: instant 7 . . . 55
5.14 Non-symmetric-hourglass shape simulation: instant 1 . . . 55
5.15 Non-symmetric-hourglass shape simulation: instant 2 . . . 55
5.16 Non-symmetric-hourglass shape simulation: instant 3 . . . 56
5.17 Non-symmetric-hourglass shape simulation: instant 4 . . . 56
5.18 Non-symmetric-hourglass shape simulation: instant 5 . . . 56
5.19 Non-symmetric-hourglass shape simulation: instant 6 . . . 57
5.20 Non-symmetric-hourglass shape simulation: instant 7 . . . 57
5.21 Configuration 1 . . . 57
5.22 Configuration 2 . . . 57
5.23 Configuration 3 . . . 58
5.24 Configuration 4 . . . 58
5.25 Configuration 4 at launch acceleration: instant 1 . . . 58
5.26 Configuration 4 at launch acceleration: instant 2 . . . 59
5.27 Configuration 4 at launch acceleration: instant 3 . . . 59
5.28 Configuration 4 at launch acceleration; instant 4 . . . 59
5.29 Configuration 4 at launch acceleration; instant 5 . . . 59
5.30 Configuration 4, in harmonic motion: instant 1 . . . 60
5.31 Configuration 4, in harmonic motion: instant 2 . . . 60
5.32 Configuration 4, in harmonic motion: instant 3 . . . 60
5.33 Configuration 4, in harmonic motion: instant 4 . . . 60
5.34 Configuration 4, in harmonic motion: instant 5 . . . 61
5.35 Configuration 4, in harmonic motion: instant 6 . . . 61
5.36 Sketch of the section revolution of configuration 4 (Dimension in mm) . . . 61
5.37 Tank complete design . . . 61

6.1 . . . 65
6.2 . . . 66
6.3 Risk Assesstment . . . 67

7.1 First Launch . . . 69
7.2 Pitch angle time history . . . 70
7.3 Altitude trend First Launch . . . 70
7.4 Second Launch . . . 71
7.5 Third Launch . . . 71
7.6 Data acquisition Third Launch . . . 71

Abstract

The present work aims at describing the design process of a water-propulsion design for the 2023 edition of the Airbus Sloshing Rocket Workshop.
The developed design is presented. The configuration involves an aft-located wing and a down-loading tail. The setup is statically stable and was chosen among others to contemplate a sloshing tank in a fore location and to keep a margin against the center of gravity shifts due to sloshing.
A Micro Controller Unit (MCU) measures the instantaneous elevation angle. The tail is movable according to the input it gets from the MCU. The goal is to have a pitch-up trend thanks to a downloading aft-located tail to enhance range and flight time performances. In the end, conclusions can be drawn from the current project.

Sloshing is a sensitive issue for aerospace applications since plane stability depends upon the position of the center of gravity. Moreover, liquid sloshing in tanks may be detrimental due to inertia forces exerted on structures or because of the decrease in performance in devices involving liquid storage (i.e., injectors in space propulsion applications).
In the application presented here, water could constitute a wide fraction of the entire mass due to budget limitations.

The first design choice was optimizing the quantity of water for propulsion to limit the sloshing mass. The propulsion system comprises $0.7\,\mathrm{L}$ of water.
Furthermore, a half-hourglass shape was designed to limit the motion of water inside the tank and to have a shift in the horizontal location of the center of gravity, allowing higher stability in the gliding phase of the flight.

To conclude, tank geometry was refined. A system of baffles was designed and distributed along the tank to optimize the flowing of water, accounting for the necessity of preventing mass sloshing and reducing inertia forces on the tank (also on baffles) and granting structural integrity.

ABSTRACT

Introduction

For centuries, one dream of Humankind was the exploration of space - leaving Earth's atmosphere and beyond. To realize this dream, engineers face daily new challenges, one of the most hindering is the **sloshing phenomenon**.

Sloshing background

Sloshing means any movement of the free liquid surface inside partially filled containers.
Free liquid surface, depending on container shape and excitation characteristics, may experience a range of motion (simple planar, nonplanar, rotational, etc.). When it comes to sloshing effects, the major issue consists of the liquid-container interaction, since the free surface can exhibit fascinating types of motion in the form of energy exchange between interacting modes. Modulated free surface occurs when the free-liquid-surface motion interacts with the elastic support structural dynamics in the neighbourhood of internal resonance conditions.

Estimation of hydrodynamic pressure distribution, forces, moments and natural frequencies of the free-liquid surface is, therefore, necessary to have a grasp on sloshing direct effect on the dynamic stability and performance of moving containers.
Mechanical models such as mass-spring or pendulum systems are usually used to model the sloshing mass dynamics while analytical solutions are limited to regular geometric tank shapes such as cylindrical or rectangular.

Some relevant theoretical considerations of liquids sloshing in closed containers are traced back to Lamb who presented the mathematical description of oscillating surface waves in cylindrical tanks based on the potential theory using Bessel functions.

Based on his approach, he could identify different sloshing modes showing a strong dependency on the excitation frequency.

Scope and objectives of the project

The aim of the project is to develop a rocket design incorporating mechanisms to manage the dynamic forces introduced by the sloshing water to maximize its range, time of flight and liquid payload capacity.

The first phase of the competition took place from March 2023 to May 2023. With regards to the AirSloths project, it stems from passionate students with different backgrounds who gathered up to carry out a long-term project as well as give valuable contributions.
The team had the mission of coming up with an innovative, non-conformal and challenging design that could integrate various technologies. The purpose was to put into practice some of the academic knowledge and get hands-on experience related to design and control fields.

Team organization

The team is composed of six people, that were dynamically assigned to different tasks according to personal interests and background.
With regards to task assignment, table 1 may summarize it.

Member	CD	DD	DO	TCC	SA	M
Maria Mattiello (T.L.)	■			■		■
Pierluca De Felice	■			■		■
Vincenzo Junior Di Rosa	■	■	■			■
Piercarlo Mirto	■	■	■			■
Maria Giulia Monti	■	■			■	■
Gabriele Salomone	■	■			■	■

Table 1
CD: Concept Desing
DD: Detail Design
DO: Design Optimization
TCC: Tail control code
SA: Sloshing analysis
M: Manufacturing

Figure 1

Methodologies employed

Analysis and design stages followed simple iterative cycles based on a starting solution and simulation outputs highlighting upsides and downsides concerning baseline and previous steps configurations.
About launch base design and construction, preliminary experiments were carried out with a pressurized bottle at the beginning of the project. This, coupled with the use of online simulators, offered valuable insights into design feasibility. Additionally, it gave experimental qualitative data on the effects of filler level and pressure on propulsion performances.
As for plane design, a baseline configuration was developed by brainstorming and qualitative considerations on the desired flight path. Alternatively, `Digital DATCOM+` codes were implemented to extract the stability/aerodynamic performances of the proposed configurations. Subsequently, iterations were performed by studying configuration effectiveness by the means of a `Matlab` code simulating the 3 DoF dynamics of a plane launched from a vertical attitude concerning the ground. Iterations regarded also optimization of the objective flight path and control law.

Sloshing simulations were carried out with `Ansys Fluent`. A baseline tank configuration was developed by studying the effect of tank dimensions on inertia forces and natural frequencies of sloshing modes. Iterations regarded flow visualization with specific tank shape/baffles distributions. Shape and baffles effects, once extracted, were shared with the team and discussed; this led to new configurations.

Requirements

List of requirements		
	Mandatory	Optional
Weight	Maximum take-off weight:<5 kg (including liquids)	
Dimensions	Maximum height 1.5 Maximum wingspan 2.25	
Propellant Tank	Tank Pressure: $< 10\,\mathrm{atm} = 147\,\mathrm{psi}$ Propellant: water Total mass of propellant less than the mass of sloshing liquid	Number of tanks can exceed 1 Reliable propellant discharge mechanism
Sloshing tank	1 unpressurized sloshing tank containing a minimum of 500 ml and filled halfway Baffles cannot cover more than50% of its cross-sectional area Sloshing tank designed to dampen fluid motion	
Launch and flight	The rocket must be launched vertically (launch angle: 85 degrees) Controlled rocket descent Re-launch within 5 min No parachutes allowed	
Costs	Cost of off-the-shelf parts<300€ Total cost<500€	

Figure 2

Part I

The aircraft model

Chapter 1

Concept design

The concept design was born around the need for the aero model to perform a flight characterized by two fundamental phases:

(1) Rocket phase;
during which the aircraft performs an almost vertical (85° from the horizontal reference plane) ascent motion. This phase quality is described by the maximum height achieved, $\mathcal{H}$.

(2) Glider phase;
where the motion turns horizontal. This phase quality is described by the maximum range achieved, $\mathcal{R}$.

The overall mission is evaluated also in terms of endurance, $\mathcal{E}$.
This crucial point brought to life the concept that will survive for all the aircraft development: a switching mechanism that ensures the maximum efficiency for both phases.

1.1 Flight path

If the aircraft has no aerodynamical properties (i.e. can't develop lift or pitching), it behaves like a blunt, rigid body subject to the sole gravity force. This means that its center of mass trajectory is a parabola, a path that can be assumed as a limit case. This provides the minimum values that $\mathcal{H}$, $\mathcal{R}$ and $\mathcal{E}$ could assume.
Of course, the aim is to exploit aerodynamic forces and moments to improve the flight parameters. This introduces the necessity of a pair of fixed wings and a consequent horizontal tail.

Hence, the focus shifted to choosing the best flight path. After various brainstorming sessions, the team came to the one in figure 1.1. It is divided into three sections:

- Trait 1: *Ascent phase*;
  where the model aircraft is in *rocket mode*. It lasts from the take-off until the reach of the apogee.
- Trait 2: *Slide phase*;
  during which the aero model descends and gains velocity due to its free fall.

- Trait 3: *Gliding phase*;
  where the model-aircraft is in *glider mode* properly. This phase must start with a positive pitching motion which smoothly changes its velocity direction in a "spoon-like" path.

The switching points between a section and the following one are key points for the mission. They are:

- Point 1-2: the apogee.
  The model aircraft must put the nose down to not stall.

- Point 2-3: the inflection point.
  Here, the pitching speed changes sign.

They delimit the *Sliding phase*.

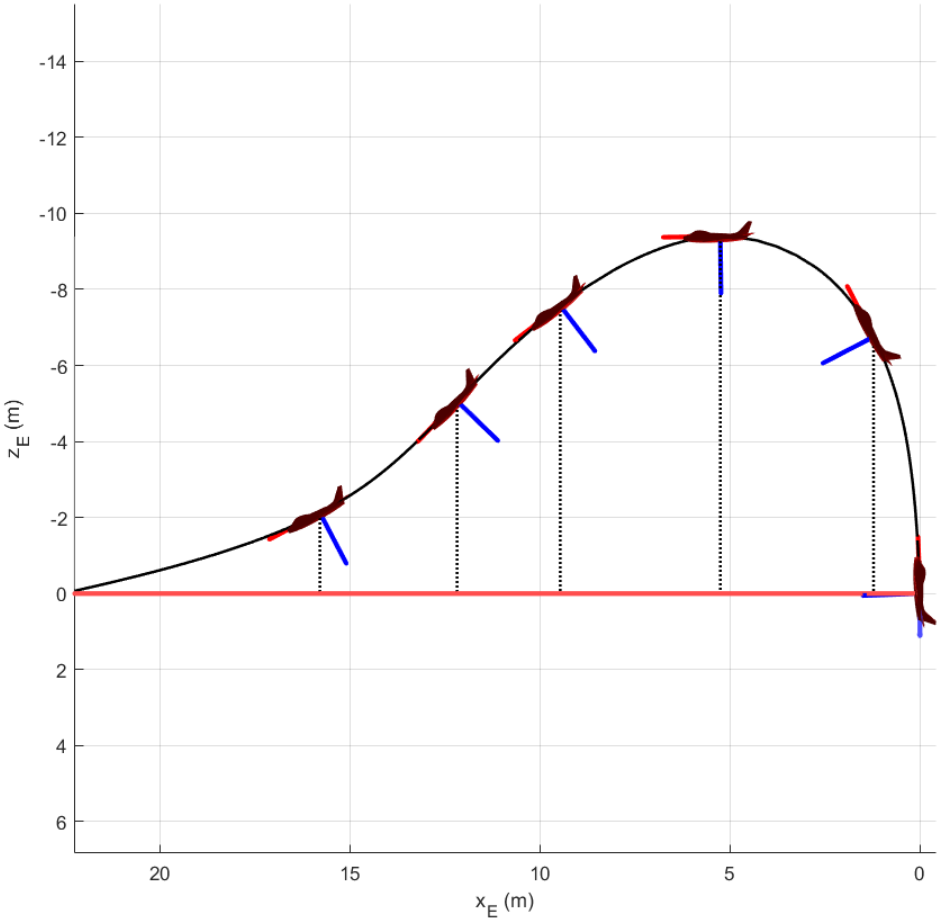


Figure 1.1 Expected Trajectory

1.1.1 Rocket mode

In this configuration, the model aircraft runs precisely along the parabolic trajectory. Therefore, no horizontal tail is needed. A vertical tail is instead necessary to govern the lateral-directional behavior, as any asymmetry is not desired in the traveled curve. Indeed, this would mean that a percentage of the global kinetic energy is wasted in a lateral component, which could instead be used to augment $\mathcal{R}$ and/or $\mathcal{H}$.
Hence, the *rocket mode* has a wing surface and a couple of vertical tails.

1.1.2 Glider mode

On the other hand, the *glider mode* needs a horizontal tail to ensure longitudinal stability. Hence comes the idea of how to merge the two requirements: a variable dihedral angle aerodynamical surface.
This idea also allowed the team to conceive a control-law concept. Indeed, if the dihedral angle can change, so can the horizontal tail lift aliquot. Programming the rotation mechanism opportunely, there is the possibility to exploit only a part of the force produced by the couple of semi-wings (see figure 2.3).

1.1.3 Mission programming

The three traits previously introduced are a function of the aerodynamic and inertial behavior of the aero model. In the first one, the aircraft model must perform a slow negative pitching motion which ensures that, once the apogee is reached, the aircraft does not risk stalling. This direction of rotation should start reducing its intensity in the second trait purposefully not giving the aircraft an abrupt transition. To do so, the tail must start opening during this phase, which concludes when the *glider mode* is fully retrieved. Finally, the *glider mode* takes action. The next step is establishing the specifics of Trait 2 (i.e. when the tail completes its opening) that effectively increment the range and endurance. Until now, it is known that the aero model starts the maneuver with a negative pitching speed which must reduce its intensity to start gliding correctly. In conclusion, the V tail must be a counter-lifting surface: At its opening, it develops a moment of positive pitching responsible for the required intensity reduction of the pitching velocity and its sign change. It must act to bring and keep the model aircraft to a horizontal asset.

The presence of Trait 2 may be counter-intuitive. Its necessity is explained considering that the velocities acting during the mission are moderately low (Mach number around 0.05). Hence, when the aircraft reaches the maximum altitude, it almost certainly will not be able to produce the required lift. The sliding phase allows the model aircraft to gain at least the quantity of dynamic pressure necessary to balance its weight force.

Chapter 2

Detailed design

Sources: [1]

2.1 Aircraft Design

2.1.1 Main design and adopted solutions

The previous chapter explained the need to maximize the dynamic pressure around the wing during the *gliding phase*. Of course, the possibility to already start with a given amount of horizontal velocity provided by the thruster would have been an extra gear.
The first solution proposed to give the aircraft such horizontal speed was a thrust-vectoring mechanism, moving the water thruster during its emptying phase. However, after some tests in which only bottles were launched, it turned out that the emptying time was too brief to do that. So, the choice fell on giving the thrust an eccentricity to develop a pitching moment right from the start. In that way, the rocket immediately gains an angular velocity which lets a portion of the thrust be directed horizontally. To do that, the thruster bottle has been placed on the top of the fuselage to have an arm between the thrust and the barycenter equal to at least the radius of the bottle itself.
Another solution to augment the eccentricity is shifting the center of gravity itself towards the aircraft's belly. Hence, the team decided to place both the sloshing tank and the vertical plane on the underside of the fuselage, as both possess a significant static moment. Indeed, the tank has a large mass while the vertical tail has a wide distance between its center of gravity and that of the whole aircraft.
Once decided that these two components should have been under the fuselage belly, the following step was to choose where in the fuselage to place them. The choice fell on placing the vertical tail in the rear because in the *gliding phase*, as previously said, it will become a reverted V tail. Instead, the Sloshing tank has been placed under the nose of the fuselage for reasons that will be explained further.

The fuselage has been designed accounting, other than the structural function, for the stresses it would have undergone at the take-off and during the landing. So the choice fell on a square-section hollow fuselage $1.35\,\mathrm{m}$ long, made of poplar wood $4\,\mathrm{mm}$ thick and with a

3 cm side; this permits easy attachments of the other components because it offers straight surfaces that allow gluing and interlocking.

Wings

The pair of wings has been devised to maximize the lift generated during the gliding phase. Here are going to be described the chosen airfoil and the shape, while the position and the dimensions will be later treated in section 2.2.

First of all, it has been set that the plan area would have been rectangular because it provides the maximum lift for a given wingspan. This choice also proved successful because it significantly simplified the construction of the wing itself.
The next step has been to decide on the airfoil. At first, the idea was to use symmetric profiles because they provided a wide range of stability during the *gliding phase* and also because they generate little lift during the rocket phase, which would have ensured a more easily predictable ascent. However, later during the designing process, studies demonstrated that this airfoil didn't provide the needed lift during the *gliding phase*. Hence, the team searched the literature for low Mach number airfoils looking for the most efficient ones, [1]. Among these, the only NACA airfoil was the NACA 6409. Since the use of `Digital DATCOM+` in the Iterative cycle (which already contains a database for this series of airfoils), the choice channeled on this one.

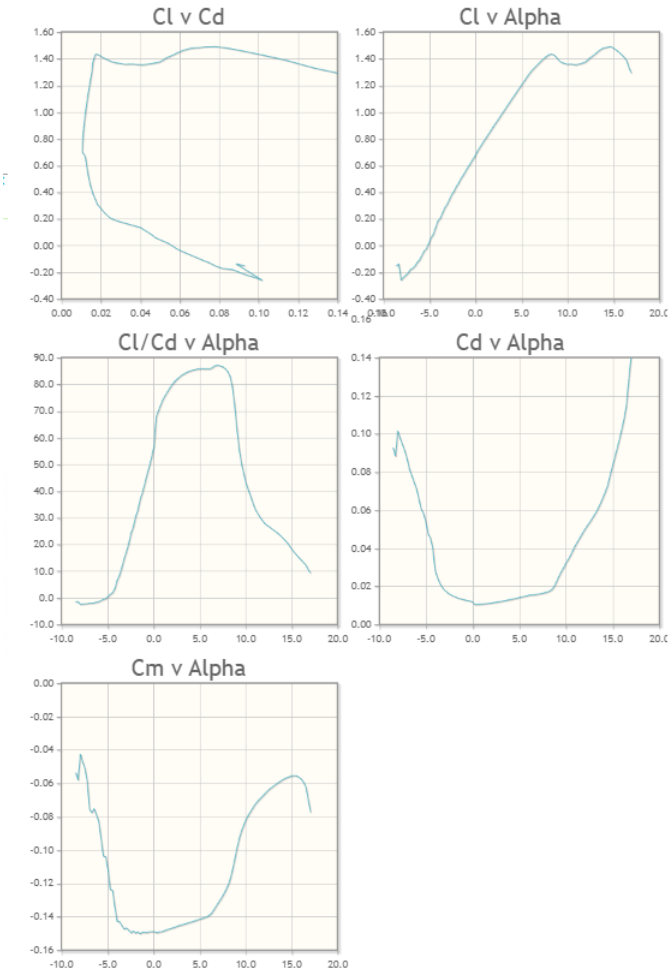


Figure 2.1 NACA 6409 Polar graphs

The wings were devised to have a filled section since it is the best choice to augment the torsional stiffness, and also it permits the interlocking with the fuselage using two spars. These

two spars hold the wings together as they are passed through the fuselage and are embedded half in the right wing and half in the left wing, each reaching half of the respective semi-wing span.

Since its start, the designing process assumed that the aerodynamic center of the wing has been placed behind the center of gravity of the model aircraft. This choice was made to provide a pitching moment at the trajectory apogee. Here, the model aircraft is still in rocket mode. Hence, since there is no horizontal tail, the only forces that can contribute to the pitching moment are the lift and the weight force, which will act like a couple, resulting in a robust negative pitching moment. This is necessary to put the nose down and to reach the correct attitude for the sliding phase of the flight path.

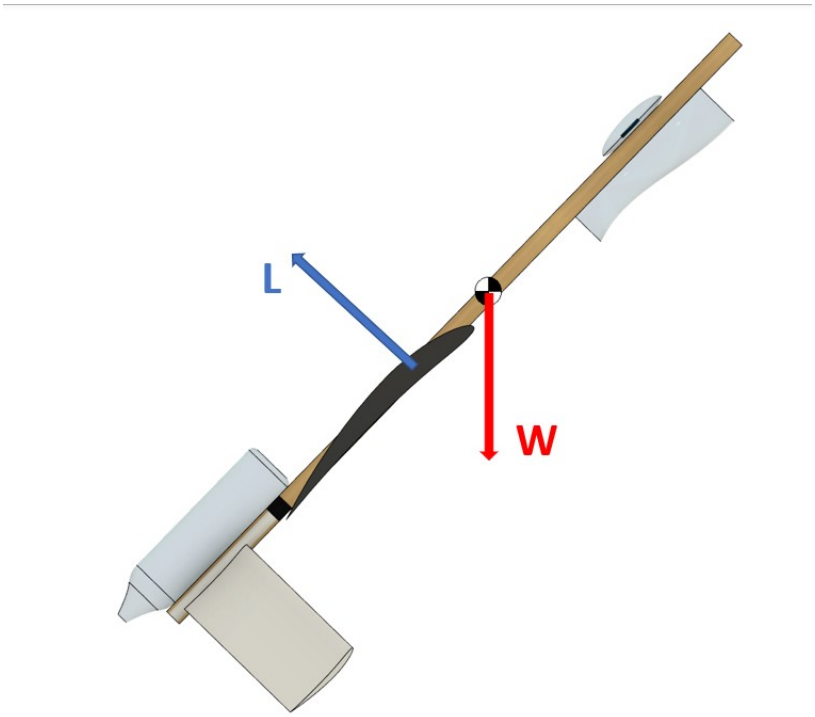


Figure 2.2

To augment the couple generated by these two forces, the better solution was to increase the arm between them, shifting the barycenter as near as possible to the nose. This leads to placing the tank of the sloshing fluid under the nose of the fuselage and the motherboard with its battery on the top of the nose. The idea of shifting the center of gravity forward to augment the couple in this phase and the controllability in the *gliding phase* has been central in the design of the Sloshing tank too.

Sliding phase

Once in the sliding phase, the aircraft accelerates toward the ground because of its free fall. Meanwhile, it is pitching down; hence to reduce the intensity of the pitching moment, it will gradually activate the mechanism that transforms the vertical tail into a reverted V tail that will generate a downforce, which produces a positive pitching moment.

Gliding phase

Since it is shaped to produce downforce, the reverted V tail will produce two forces perpendicular to the wings, as shown in figure 2.3.

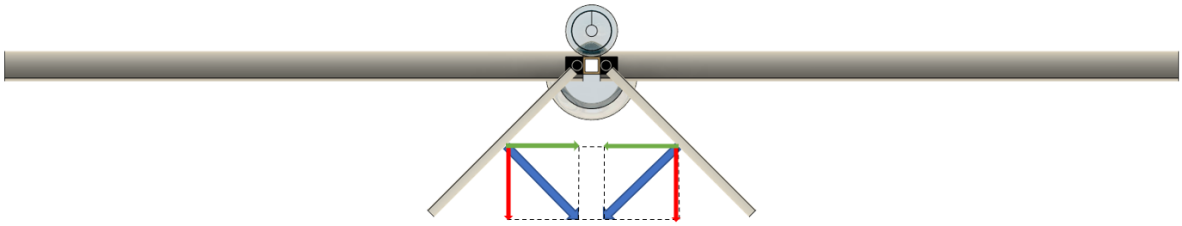


Figure 2.3

However, only the vertical components will influence the pitching stability, while the horizontal ones will elide each other. The idea is to vary the vertical component of the downforce by changing the dihedral angle of the reverted V tail. By doing that, it can actively control the horizontal attitude during the descent in the *gliding phase*. In addition, the idea of placing it under the belly of the aircraft is another time winning as the downwash of the principal wing reduces the angle of attack on the airfoil, resulting in a stronger downforce produced by the reverted V tail.

The requirements said until now took the team to concept an aircraft model that looked similar to the MQ-1 Predator, figure 2.4.



Figure 2.4 Credits: https://www.military.com/equipment/mq-1b-predator

2.1.2 Possible applications

By no means the solution adopted for the aircraft tail opens new horizons on how we can see aerodynamic actuators. Indeed such an aerodynamic surface allows an aircraft to significantly vary the lift produced without losing performance. Furthermore, this solution provides this possibility without any risk of stall for the airfoil because the airflow around the wing never varies. For example, a military aircraft could completely remove the stability contribution given by the horizontal tail, meaning complete instability of the aircraft which allows very

low-radius maneuvers. Here it is used to have instability on the apogee and then to actively control the gliding phase.

2.2 Iterative cycle

Figure 2.5 shows the philosophy adopted in this design phase. Taking as initial input the data relative to the PREDATOR-like architecture, a flight simulation ran in `Matlab` took place. The aircraft was modeled using the software `Digital DATCOM+`, more on appendix (section A.2). Once seen the resulting trajectory, the architecture was changed and another simulation took place.
The software `Digital DATCOM+` needs the aircraft's center of mass position to evaluate its stability. However, this property isn't fixed yet, hence the birth of the iterative loop. The variables that changed during the iterative cycle were, for both the wing and the horizontal tail, the following:

- root chord;
- span;
- apex position.

2.2.1 `Matlab` code

The code works as follows. Firstly, it describes the model aircraft as a set of material points which are the various components. Their position is defined in a constructive reference frame (origin on the aircraft nose, x-axis oriented backward, and z-axis oriented downward); their mass is known or estimated once given the material density and the object volume.
Together with the total mass of the aero model, the code also estimates the moments of inertia, which are necessary for the flight simulation. The aerodynamical properties are previously downloaded from the output file of the `Digital DATCOM+` codes (a series of eight codes for the different dihedral angles).

In the flight simulation, an `if()-else()` statement simulates the control law.

Once, calculated the center of gravity of the obtained aircraft model these data were set as an input for the eight `Digital DATCOM+` configurations, and the stability derivatives were taken as output and imported in the second part of the `Matlab` code, which solves the Full System of Flight Dynamics Equations, choosing the aircraft configuration according to the angle of elevation. The third part of the `Matlab` script displays the solutions of the differential equations, providing data on attitude, altitude, and angles of elevation and attack at every moment until the landing; furthermore, it provides the Range and time of flight. Analyzing the results given by the Matlab code the iterative cycle took place.

2.2.2 Mindset

The cycle was executed in such a way as to affect the simplicity of the model's construction as little as possible. This means that:

- both the wing and the horizontal tail are straight and with constant section (this also helps in the mathematical modeling);
- fuselage is of constant section;
- The aerodynamical surfaces do not have any twisting or incidence angles.

These properties, even if can be changed to enhance the aircraft's performance, were never touched.

Simulations were also used to understand what was the ideal range for the dihedral angle values of the horizontal tail, to give them as input for the coding of the control law.

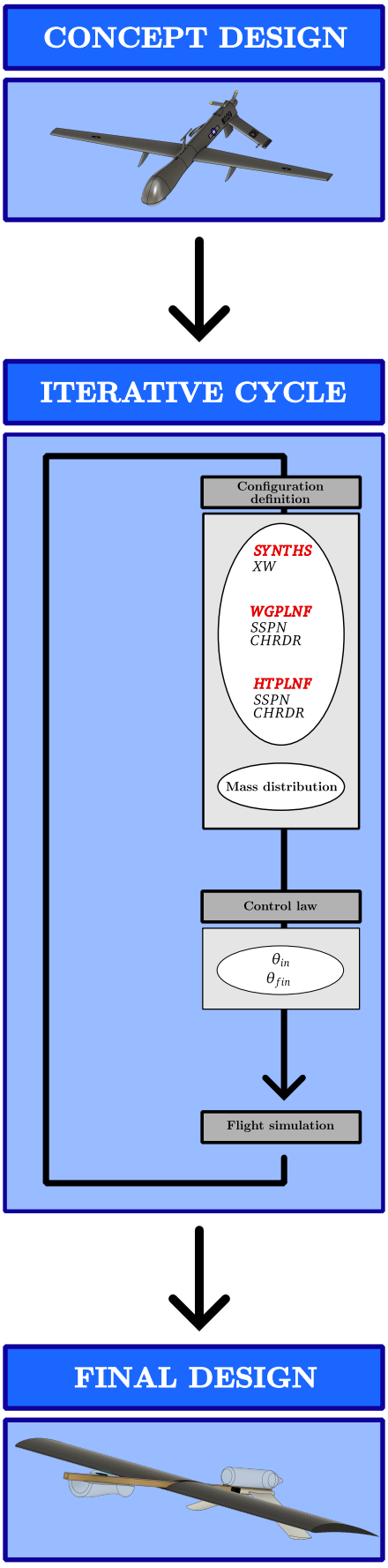

CONCEPT DESIGN
ITERATIVE CYCLE
Configuration definition
SYNTHS
XW
WGPLNF
SSPN
CHRDR
HTPLNF
SSPN
CHRDR
Mass distribution
Control law
θ_{in}
θ_{fin}
Flight simulation
FINAL DESIGN

Figure 2.5



Figure 2.6

2.2.3 Results

In the first iteration, the wings had a chord of $0.35\ \mathrm{m}$ and a $1.6\ \mathrm{m}$ wingspan. Simulations showed that this wing configuration didn't produce enough lift to permit a gliding phase. After several simulations, the aircraft took:

- a $10\ \mathrm{cm}$ increase of its wing root chord ($0.45\ \mathrm{m}$);
- a $40\ \mathrm{cm}$ increase of its wingspan (now $2\ \mathrm{m}$).

The resulting architecture recalls an ancient flying reptile: the *Pterodactylus*; hence, playing with the word Predator the aircraft model was baptized Pteradon.

The iterative cycle brought the expected time laws of linear speed, pitching velocity q, and elevation angle θ. They are shown respectively in figures 2.7, 2.8a and 2.8b.

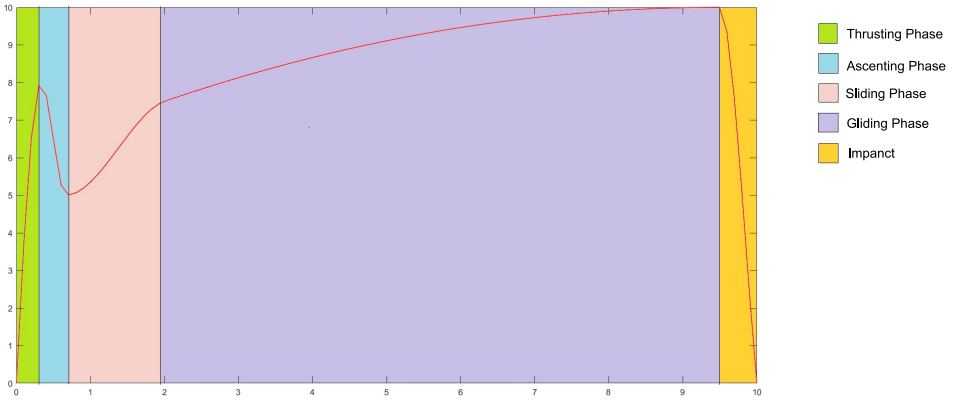
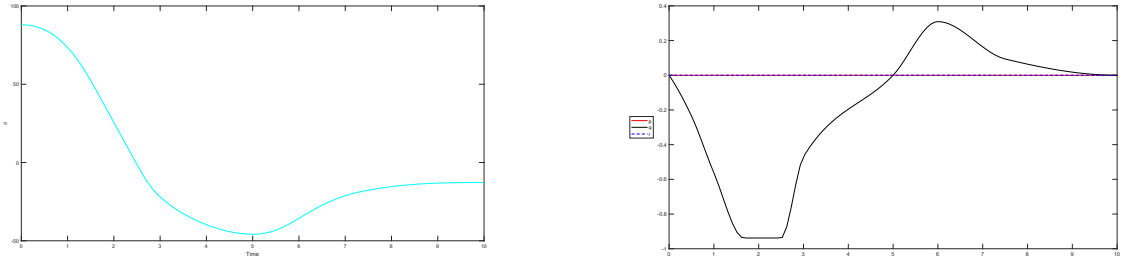


Figure 2.7 Velocity to time graphic



(a) Theta to time

(b) q to time

Figure 2.8

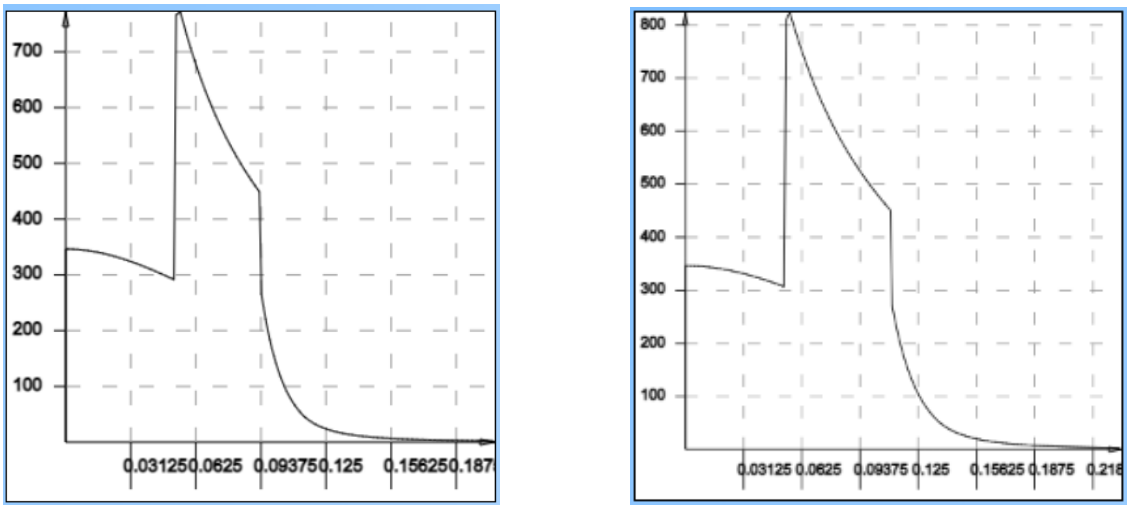
2.3 Propulsion

Studies on the propulsion system have been conducted since the very early stages of the designing process. This provided the team with an initial insight into the phenomenon of water propulsion and highlighted the forces and stages involved during a launch. The very first tests and research were made to prove that ordinary carbonated drink bottles could withstand a pressure of 10 bars and gave a positive result. These tests have been made using the first version of the launch base, with no launch tube and a different withholding mechanism, since its only purpose was to test the tank limits. Among several possibilities, after comparing all the bottle shapes available on the market in our region, two main options survived, shaped as shown in figure 2.9.



Figure 2.9 Bottle Comparison

Between these two possibilities, it was assessed that the sharp end of bottle B could induce a loss of dynamic pressure at the nozzle, so bottle A was chosen as the final tank for the propulsion system because of the smoother shape. Studying literature about water propulsion, it has been asserted that the optimal filling of the tank would have been 1/3 of the total volume. Once decided on this, the next step was to choose the tank volume. In our country two formats are commercialized: 2 liters and 1.5 liters; since the filling would have been 1/3, and considering that the propulsion mass of water must not exceed the water contained in the sloshing tank, studies about the Trust to TOW were made. Indeed, considering that the empty weight of the model aircraft is $2.7\,\mathrm{kg}$, the options were to add to this weight $1.4\,\mathrm{kg}$ ($0.7\,\mathrm{kg}$ of propulsion water and $0.7\,\mathrm{kg}$ of sloshing water) in the $2\,\mathrm{L}$ thruster configuration, or to add $1\,\mathrm{kg}$ in the $1.5\,\mathrm{L}$ configuration; both were possible since the maximum wet mass is 5 kilograms. To investigate these two configurations, online water rocket simulators have been used. In particular, the Thrust curve and emptying time have been evaluated. The result of this investigation sees the $2\,\mathrm{L}$ thruster configuration winning due to its higher thrust-to-weight ratio, and because of the longer emptying time that permits to exploit the pitching moment caused by the eccentricity of the Thrust as explained in section 2.1.1.



(a) 1.5 Bottle **(b)** 2L Bottle

Figure 2.10 Thrust to time graphics

2.3.1 Launch base

As previously said, a first launch base was used to test the bottles in the early stages of the project, but later it was completely redesigned to maximize its efficiency. The final configuration consists of a horizontal platform to which a Clark's cable tie system attached to the pressurization tube and a support rod for the rocket are attached. The Clark's cable tie system provides the possibility of a high-efficiency withholding mechanism and an immediate release, other than the possibility of having a launch tube. The launch tube adds a Thrusting phase to the ascent, to maximize the length of this phase and the addictive thrust itself, a $30\ \mathrm{cm}$ long launch tube has been designed, and the bottle is only $3\ \mathrm{cm}$ longer. The pressurization is actuated by a bike pump, inflating air through a valve located at the end of the horizontal section of the tube. The valve used is a race bike pneumatic valve which, if necessary, can be pressed to release air and depressurize the tank, ensuring the safety of people around the launch base.

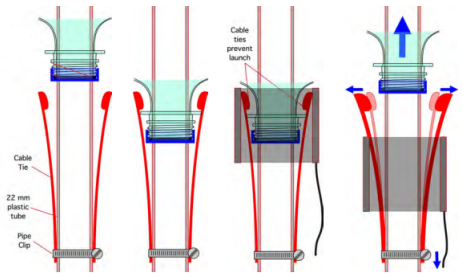


Figure 2.11 Clark's cable tie system

The support rod is designed to guide, together with the launch tube, the rocket in the lift-off, it also allows us to precisely control the inclination of the launch to respect the requirements.

2.3.2 Launch procedure

The launch procedure has been devised to permit all the pre-flight operations, while maximum safety and efficiency are ensured.

- Phase 1: Placing the rocket upside down and filling the tank with $0.7\,\mathrm{L}$ of water.
- Phase 2: Horizontal placement of the launch base and shifting of the rocket in a horizontal position,
- Phase 3: Insertion of the launch tube and support rod in the bottle nozzle and the fuselage hollow tail, respectively.
- Phase 4: Calibration of the avionic.
- Phase 5: Vertical placement of the rocket and its launch base.
- Phase 6: Check for any possible issue and leak.
- Phase 7: Beginning of the tank pressurization.
- Phase 8: Completion of pressurization at 9 bars.
- Phase 9: Final check and Go For Launch.
- Phase 10: Activation of the Releasing Mechanism.
- Phase 11: Take-off!

Chapter 3

Control System Implementation

To control the descent of the vector, it was necessary to implement an electronic system. The idea is to actively stabilize the aircraft by changing the opening angle of the tailplane to dump the effects of the moment caused by the sloshing and to keep the model aircraft in a stable position. Given the hardware capabilities, it was also possible to add some other functions to enable the altimeter and data saving on an external SD Card.

3.1 Component description

The core system used for collecting and processing data is the NUCLEO F401RE, provided by our sponsor STMicroelectronics[1]. This development board is one of the STM32 microcontroller series. More specifically, it is an STM32 Nucleo-64 board. This low-cost, user-friendly development platform has unlimited unified expansion capabilities and integrated debugger/programmer functionalities.
The expansion board used is the X_NUCLEO_IKS01A3, a motion MEMS (MicroElectroMechanical System), and environmental sensor evaluation board system. Thanks to it, the team could measure angular velocities and accelerations and track distance to the ground. The system was integrated with an SPI Reader micro memory SD module to collect flight data during the test and with two AZDelivery MG996R micro digital servo motors to move the tailplane. Everything was powered by a 2s Lipo Battery ($7.4\ \mathrm{V}$) of $2700\ \mathrm{mA{\cdot}h}$, with a C-rating of 25C.

[1] Official site: https://www.st.com/content/st_com/en.html.

3.2 IDE

Mbed Studio is a free IDE for Mbed OS applications and library development, including all the dependencies and tools you need in a single package so that it is possible to create, compile and debug Mbed programs directly on the desktop.
Mbed Studio is a local development environment for Mbed OS 6 programs written in C/C++. This IDE can autodetect the board used when plugged in. It autonomously flashes the code directly to the platform. It is equipped with a pre-defined debug profile. All these features make the sketching process of user programs more intuitive and less demanding. The code can be built with Arm Compiler 6, which is included free of charge in the IDE and has no restrictions on code size or optimization levels.

3.3 Libraries

X-NUCLEO-IKS01A3 MEMS Inertial & Environmental Sensor Nucleo Expansion Board Firmware Package

The firmware package includes the Drivers and the Support package necessary for using the MEMS Inertial&Environmental Sensors installed on the expansion board X-NUCLEO-IKS01A3, more at the link:

https://os.mbed.com/teams/ST/code/X_NUCLEO_IKS01A3//file/aea924ef2645/XNucleoIKS01A3.h/.

Class X_NUCLEO_IKS01A3 represents the expansion board with the same name and gives access to its sensors. More specifically, the following sensors give the necessary inputs in the code developed here:

- the LPS22HH MEMS Pressure and Temperature Sensor;
- the LSM6DSO 3-Axis Accelerometer and 3-Axis Gyroscope.

IMU filter

In 2009 Sebastian Madgwick developed an Inertial Measurement Unit (IMU) and an Attitude and Heading Reference System (AHRS) sensor fusion algorithm as part of his PhD thesis at the University of Bristol, [2]. They are provided at the link:

https://os.mbed.com/users/aberk/code/IMUfilter/.

The IMU filter gets data from tri-axis gyroscopes and accelerometers to compute the Euler angles in real-time. The benefits of the filter include:

- its computational inexpensiveness;
- its effectiveness also at low sampling rates, e.g., 10 Hz.

The low computational load and the ability to operate at different sampling rates make it possible to use IMU sensor arrays in real-time applications of limited or processing resources that require high sampling rates.

FATFileSystem and SDBlockDevice

These two libraries communicate with the SD card and permanently store data in it. The File Allocation Table (FAT) file system is an established disk-oriented file system that can be found on Mbed OS, Windows, Linux, and macOS. Due to its age and popularity, the FAT file system has become the standard for portable storage, such as flash drives and SD cards.
Mbed OS SD card block device allows programs to read and write data to flash storage cards using the standard POSIX File API programming interface. To enable the SD card, an `mbed_app.json` file should be added to the program project with the following command to activate the SD component: `target.components_add : ["SD"]`

3.4 Program structure and basic idea

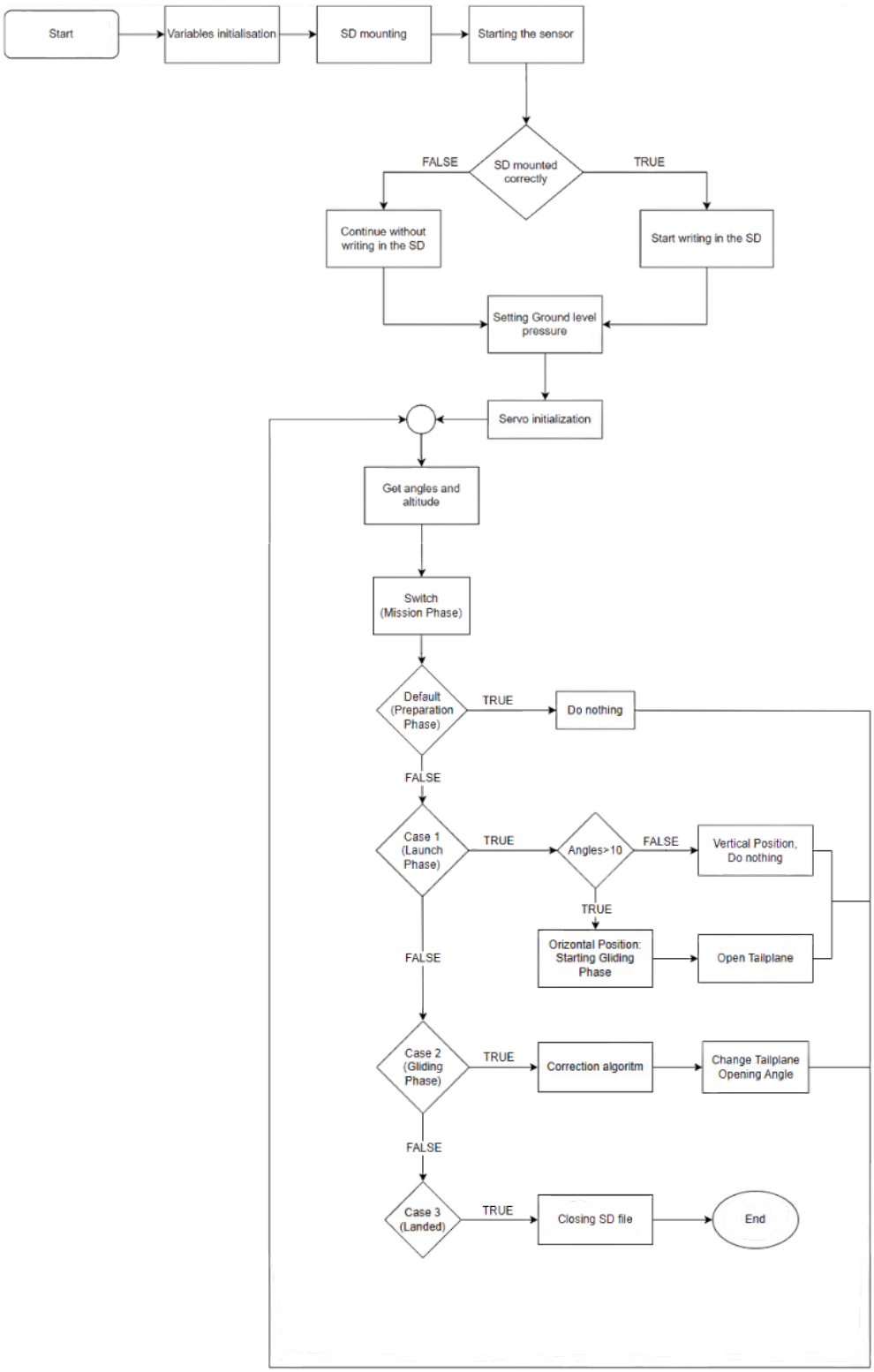


Figure 3.1 Flowchart of the entire code

Figure 3.1 shows the flow chart. The program can be divided into two parts: the program initialization, which includes the libraries and the definitions necessary to run the program, and the main, which represents the core of the program itself. This report focuses on the main function since it describes the idea behind the code and all the processes happening during

the run.

The main itself can be divided into two parts:

- the hardware setup;
- the `while` loop.

The first part consists of enabling the sensors (IMU, Barometer, and SD) and the peripherals (The SD card, the servos, and the User Button) and setting the parameter for making them work properly. In this phase, the barometer (alias, the aircraft) must stay in a stable position on the ground for a few seconds so that the MCU can take multiple measurements to set the Ground Level pressure. This data will be later used in the program to calculate the altitude reached by the aircraft. Before the start of the **while** loop, the program starts a timer that will count the time during the whole mission.

3.4.1 `while()`

A `while()` loop is a control flow statement that allows a set of instructions to be repeatedly executed as long as a specified condition remains true.
In the code written, it can be divided into two main parts:

- the measurements part;
- the switch.

In the measurements part, the program gets the data from the sensors. More specifically, the IMU gives the accelerations and the angular velocity, while the barometer gives the pressure value. The IMU data are then converted into measure units that can be processed by the Madgwick filter ($\mathrm{m/s^2}$ and $\mathrm{rad/s}$). The filter is then updated, giving back pitch and roll.
The program proceeds to estimate the altitude by using the Hydrostatic Equation:

$$P - P_{ground} = -\rho\, g\, \left(h - h_{ground}\right) \ ,$$

which here assumes $\rho = 1.225\,\mathrm{kg/m^3}$ and $g = 9.81286\,\mathrm{m/s^2}$. After all, Pitch, Roll, Altitude, and Time are printed on the SD Card.

3.4.2 `switch()`

A `switch()-case()` statement is a control flow statement that handles multiple possible code blocks depending on a parameter value. In this reported case, the parameter is `phase`, used to differentiate the various mission phases.
When the code starts, the parameter `phase` is initialized to zero. When the program passes through the `switch()`, nothing happens. This is the *Preparation Phase*: it gives the team the time to set up the aircraft and prepare for the launch. When everything is ready, by pressing the User Button (blue button), the `phase_add()` function is activated, which increases the number of the parameter `phase` by one, starting the *Launch Phase*. In this phase, the program waits for the aircraft to reach a specific angle to begin the *Glider Phase*; when it gets to the angle value specified in the `if()` condition, the angles of the tailplane are set to a pre-programmed position to give stability to the aircraft, and the `phase` parameter is increased by one.

The *Glider Phase* is a flight correction algorithm that consists of a series of `if()-else()-if()` statements that determine the opening angle of the tailplane. Pitch angle values and opening angles are obtained from a series of tests.

After the landing of the aircraft, by pressing the User Button, `phase` is increased by one, getting to "case 3" or the *Closing Phase*. In this phase, the program closes the file created in the SD and enters a `while()` loop that keeps going until the MCU is turned off and back on or the program is restarted by pressing the black button.

3.4.3 Flight correction algorithm

Initially, the flight correction algorithm was built to change the tailplane opening angle linearly with the pitching angle. In particular, when the pitch angle gets to 0, the tailplane starts to open, and the opening angle increases with a more negative pitch angle (see paragraph A.2.4 of the Appendix for the exact angle).

First test

Then the launch test revealed some unexpected errors in the angle calculation. When the MCU is in a vertical position for a long time and is then affected by a strong impulsive acceleration, the values returned by the IMU filter are not reliable anymore.
During the static testing phase, the IMU filter library returned -0.0000 as the pitch angle when the MCU was positioned vertically. More tests revealed that the value was not an actual zero but extremely high in the module. Since this did not appear to be an issue in static conditions, it was not considered a significant problem. The team proceeded to the launch test.

Second test

It was necessary to do a second test to see the behaviour of the tailplane during the flight path. So, the problem was overcome by doing a trade-off between the data collected (go to paragraph A.2.4) and the video of the first test. This analysis demonstrates that when the Pteradon is horizontal, the IMU filter retrieved a pitch value of $37°$. The flight correction algorithm was changed so that the tailplane starts to open at $30°$, as shown in Appendix A.2.4. The second test demonstrated that the tailplane opened. However, it didn't give the desired effect. This is mainly due to two reasons:

- The first one is linked to the problem presented before, the error in the angle calculation. The transition between the *Launch phase* and the *Glider phase* isn't smooth as expected. Hence, the consequent change in flight attitude is not the one predicted by the simulations.

  With further investigation, this problem can be solved using a library that better implements the Madgwick algorithm or by including the magnetometer in the algorithm (AHRS: Attitude and Heading Reference System). The magnetometer implementation would also return more precise angles, reducing the accumulated error in angle calculation.

- The second one is linked to the dimension and configuration of the tailplane as explained in 7.2.

Chapter 4

Prototype Construction

4.1 Goals and Management

Prototype construction took a week, in May. Team members gathered to assemble a prototype featuring the aerodynamic and structural characteristics of the designed model, to obtain inputs related to its stability and structural resistance. More specifically, desired launch testing outputs concerned the aerodynamic capabilities of the lifting surfaces, control law effectiveness, and wing stiffness when the model is launched.

4.2 Construction Details

4.2.1 Launch base

The launch base consists of a wooden table that lays on the ground with four legs and is drilled to host a PVC launch tube that should be inserted in the propulsion bottle and an aluminum guide profile where the rear part of the fuselage is inserted.

The aluminum rod was attached vertically using bent perforated metal plates to ensure the stiffness of this structure. The bottle is anchored with the so-called Clark's cable tie system, which holds the bottle at the thickening of the launch tube created by heating the PVC. Plastic ties are used to fix the bottle in the desired position, held in place by a cylindrical plastic element. In turn, this is held in position by the spring-back force of a cut bottle, creating a joint with the thickening on the neck of the bottle.

The plastic cylinder is tied to a tape, which, when pulled, opens the mechanism releasing the bottle. At the bottom of the vertical PVC launch tube, under the wooden platform, is attached a horizontal PVC tube that allows inflating air by a 1.5 meters distance from the launch tube itself. To reduce the possibility of leaks in the joint between the two PVC tubes, a glue reinforcement has been made to maximize the efficiency of the o-rings, sacrificing the possibility of disassembly (unnecessary). The valve where the pump is attached is placed at the end of the horizontal tube. The proper pressurization is actuated by a bike pump manually operated.



Figure 4.1 Launch base

4.2.2 Fuselage

Fuselage was realized in poplar wood. Four strips with matching ends were cut from a $4\,\mathrm{mm}$ sheet and then assembled to make the strips match (Figure 4.2). To interlock pieces better, poplar strips were sanded down before assembly.
Three poplar wood blocks of the same size as the cross-section were placed inside the structure. They were located at the tip, 39% and 78% of the fuselage length. Moreover, at the bottom of the body, a hollow section was left so the guide tube belonging to the base could enter inside. At the wings' carbon fiber sticks location, the fuselage was locally drilled to lay wing sticks in (Figure 4.3). To not have shear eccentricity, one of the spars lies in correspondence with the barycentric line.



Figure 4.2 Fuselage interlocking

4.2.3 Wings and Horizontal plane

The wing and horizontal plane are made of polystyrene. Their shape was obtained by working $6\,\mathrm{cm}$-thick blocks with hot wire. In particular, the main wing is made of EPS since it's lighter, and the horizontal plane is made of XPS as its thinness does not allow to use of the EPS, which would break down.
Later, a scotch tape cover was added to the main wings to reduce the wing's surface roughness and to strengthen the wings' surface. To connect the two half-wings, a couple of carbon fiber sticks were employed with a cross-section of $4 \times 4\,\mathrm{mm}$ (rear) and $8 \times 8\,\mathrm{mm}$ (front), and with a length of one meter, figure 4.3.



Figure 4.3 Wings sticks

Shaping

The team assembled a hot wire device, shown in figure 4.4. To use it, three people are required. A first-person shall set the pace and move the central bar in the chordwise direction, while two people at opposite sides of the wing (or tail) shall direct the hot wire to extract the required airfoil shape.



Figure 4.4 Hotwire device

Horizontal plane moving system

The tailplane structure consists of two electric motors, a 3D-printed cover to protect them, two tubes, and a perforated steel plate. The body was fixed to the fuselage using epoxy glue and M2.5 screws.
The process of fastening consists of three phases. First, the motor case was fixed to the fuselage using glue. Then, two M2.5 screws were used to ensure the attachment, taking particular care to position the screws correctly so they do not interfere with the guide rod that has to be inserted into the fuselage. After that, the tube was connected to the motor, and the end was fixed using the perforated steel plate. Each tube has two holes that guarantee the fit of the wings' carbon fibers. To improve resistance, the glue has been inserted to secure the anchoring.



(a) *Structure* **(b)** *Centering*

Figure 4.5 Tailplane structure

4.2.4 Control system

At the front tip, Nucleo F401RE, X_Nucleo_IKS01A3 and Micro SD module are placed and shielded by a plastic cover. Everything is connected to a prototype board placed at the center of the fuselage. As shown in picture 4.6, the board hosts four 10 Ohm resistors (two for each servomotor) since the battery voltage is too high for them.

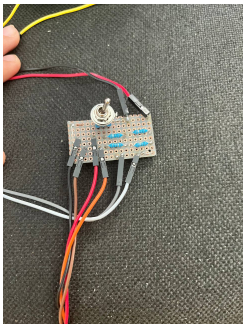


Figure 4.6 Central Circuit

In conclusion, this electronic circuit connects the power supply to all the devices used. The battery was placed under the control hardware and fixed using electric tape. Control system cables run all along the fuselage. The type of cable used was wire jumpers reinforced by using electric tape.



Figure 4.7 Electronic system final configuration (without plastic cover)

4.3 Process of Construction

The construction process can be divided into stages that may be summoned as:

1. Launch base assembling and fuselage poplar wood cutting and assembling.
2. Wings and horizontal plane shaping via hot wire technique.
3. Fuselage-Lifting surface fixing and fuselage-motors fixing.

Phase 1

In the first phase, the launch base was assembled. Subsequently, the fuselage was realized by cutting poplar wood to interlock four sheets of wood.

Phase 2

Phase 2 consisted of lifting surfaces shaped by a hot wire. Before cutting polystyrene wings and tail, airfoils were printed; cardboard pieces were cut with airfoil shapes to become a cutting guide.

Phase 3

Phase 3 consisted of residual work related to assembling. Carbon fibers stick were inserted in lifting surfaces by heating sticks of equal dimensions as the inserted one. Electrical engines were fixed to the fuselage using M 2.5 screws and glue.

Used Tools

Here is a list of the tools utilized during the manufacturing stage.

- Drill
- Brushless jig saw
- Hacksaw
- Hotwire
- Li-po battery charger
- Soldering iron
- Candle

4.4 Bill of Materials

The complete Bill of Materials is presented in 4.1 right below. The reader can find all the materials utilized for the construction with the corresponding price and supplier. All the prices are in EURO (€) and don't include possible shipping costs. The total cost of the prototype is the **406.82€**, but this total includes also the cost of spare parts in case they get damaged during the manufacturing stage.

Name	Qty	Supplier	Unit Price	Total Cost
Aero model structure				
EPS polystyrene 0.5 × 1 × 0.06 m	3	Leroy Merlin	4.32	12.96
XPS polystyrene 0.6 × 1.25 × 0.05 m	1	Leroy Merlin	13.55	13.55
Square carbon tubular 0.004 × 1 m	3	ModelToy	7	21
Aluminum square profile L 1 m x 0.008 m	1	Leroy Merlin	2,95	2,95
Square carbon tubular 0.008 × 1 m	2	ModelToy	10	20
SQUARE PIPE 0.004 m x 0.004 m	1	cmmodellismo	2,95	2,95
Poplar plywood 300 × 1.35 × 4 mm	1	Leroy Merlin	28.96	28.96
Pattex Power Epoxy Instant Mix	3	Amazon	9.95	29.8
Vinyl Adhesive Glue for Wood, Glass, Ceramics	1	ebay	6.50	6.50
2 m " PVC Tube (20 mm OD)	1	Leroy Merlin	1.9	1.9
Galvanised perforated Steel Band Tape	1	Leroy Merlin	8.75	8.75
Sandpaper	1	Amazon	5.99	5.99
Electric tape	1	Amazon	3.59	3.59
Screw and nuts M2,5 L20 mm	30	Leory Merlin	N/A	2.15
Screw M3 L 12 mm	3	Leroy Merlin	N/A	2.50
Tape	4	Leroy Merlin	1.99	7.96
PET Bottles 2 L	2	Carrefour	2.79	5.58
Electronic components				
NUCLEO-F401RE STM32 Nucleo-64	1	STM	12.64	12.64
X-NUCLEO-IKS01A3	1	STM	18.22	18.22
SPI Reader Micro Memory SD	1	AZ-Delivery	3.99	3.99
Roaring Top 2700 mA·h 2s (7.4 V) 25C Lipo Battery	1	Amazon	N/A	34.99
AZDelivery MG996R Micro Digital Servo Motor	2	AZ-Delivery	9.99	19.98
Micro Memory SD	1	Amazon	6.99	6.99
Jumper Wire	1	Amazon	9.98	9.98
Resistor 10 Ω	10	ebay	N/A	3.9
Eletric cables	1	Amazon	13.99	13.99
Prototype Board	1	RS	6.54	6.54
Launch base				
2 m PPR Tube 20 mm OD	2	Leroy Merlin	2.8	5.6
Stainless Steel jubilee type clip - 27-51 mm	4	Amazon	N/A	4.99
Inflator	1	Amazon	20.99	20.99
Bicycle Air Chamber w/ Schrader Valve	1	Decathlon	3.19	3.19
Plastic PN16 Polypropylene Elbow Fitting Pipe Tube	1	Leroy Merlin	1.6	1.6
End cap 20 mm, PN: 16 bar	1	Leroy Merlin	1.15	1.15
Hot glue and glue stick	1	Amazon	N/A	9.99
Plastic Cable Ties	110	Amazon	0.03	3.59
Square tube 1 × 0.016 m	1	Leroy Merlin	5.25	5.25
Screw and nuts M6	6	Leroy Merlin	N/A	3
Screw M4 L 20 mm	30	Leroy Merlin	N/A	1.5
Triple depth glass OD 40 mm	1	ebay	4.86	4.86
3D printed Parts (*)				
Engine mounts	2		2.97	5,94
Baffles	4		N/A	1.10
Cover for the PCB	1		3.61	3.61
Sloshing Tank bottom part (**)	1		5.81	5.81
Sloshing Tank top part(**)	1		17.32	17.32
Total cost: 406.82€				

Table 4.1 Bill of Materials and Cost Breakdown.
N/A: Not Available.
PN: Nominal pressure.

(*) 3D-printed parts prices were calculated considering the 3D printer Elegoo MARS Pro 2 and the resin Plant Based Elegoo Black which cost 25 €/kg.
(**) Due to his dimension, the Sloshing tank was divided into two parts.

Part II

Sloshing

Chapter 5

Introduction to sloshing in the design

Regulations requested the Take-Off mass to be less than $5.0\,\mathrm{kg}$. That means that once the propellant water is expelled, sloshing mass may take up a large percentage of the total weight of the prototype launched.
It is needless to say that sloshing mass deeply influences the Pteradon stability and thus design requirements. Indeed:

- the sloshing mass takes up a large fraction of the mass that significantly moves the prototype center of mass;
- the liquid exerts inertia forces on the structure mounting the tank.

To avoid these drawbacks, the design of the sloshing reduction system must be based on both reducing the motion of water and reducing the inertia forces exerted.
It was thought that limiting too much the motion of water in the sloshing tank would lead to a distributed contact between the structure and water, leading to a complex and distributed force system. Moreover, to compensate for the movement of the center of mass caused by sloshing, the design of the model involves rather aft wings and a downloading horizontal plane.

5.1 Workflow for sloshing reduction

The sloshing reduction system unfolded in three main phases:

- Conceptual design of a rocket/glider model configuration ensuring a certain margin of static stability against sloshing movement
- Optimization of sloshing mass carrying system by choosing a geometry
- Sloshing mass simulation and configuration definition

The approach that was followed is displayed in Figure 5.1

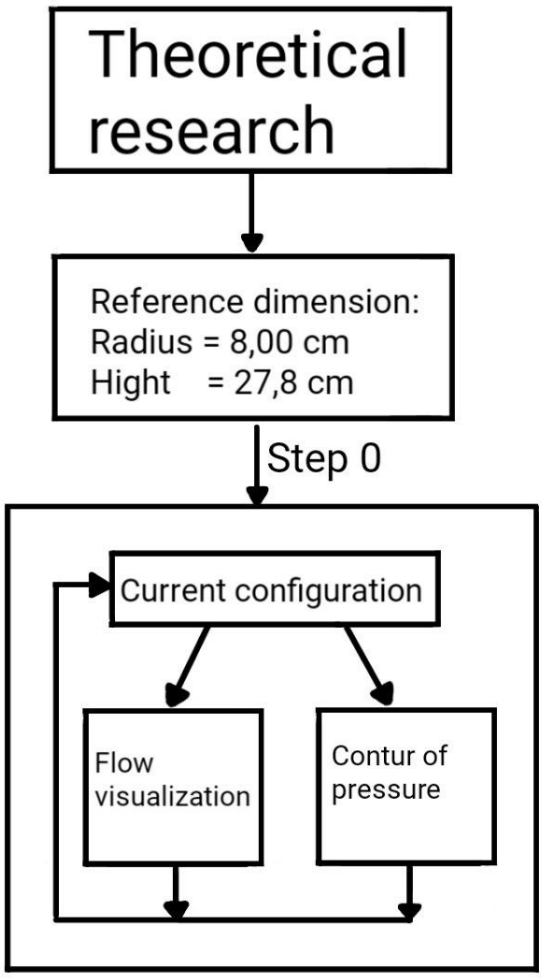


Figure 5.1 Schematization of performed task

5.1.1 Optimized design features to allow lighter sloshing issue

Here we report configuration features that were introduced to optimize sloshing challenging characteristics.

- One 2 L bottle of propellant water (fill level at 30 $\%$ was chosen as a propulsive system: this ensures a good thrust for light structures as well as leads to the possibility of reducing the sloshing mass according to regulations to 700 mL
- Both wings and (down-loading) horizontal plane are located far behind the ideal position of the sloshing bottle so to enhance the stability features of the model.
- Sloshing mass is an aft-located element of the configuration to maintain a stability margin against the Center of gravity shift due to sloshing

5.1.2 Choice of reference dimensions

Regarding reference dimensions, a starting cylinder shape of 1.4 was taken as a reference. Design should be driven by:

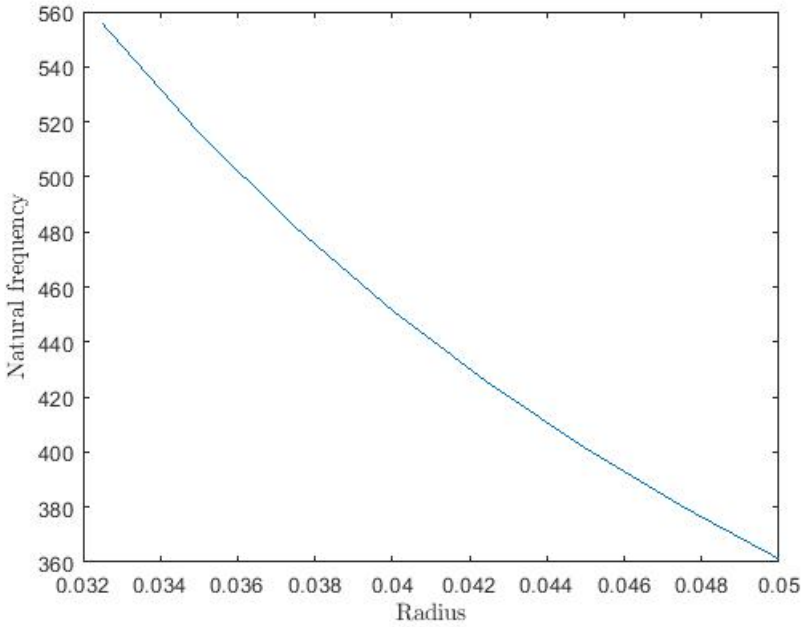


Figure 5.2 Natural frequency trend

- increasing natural frequencies
- reducing inertia forces acting on tank walls

A `Matlab` code was implemented to extract such characteristics for the cylinder shape and to find an optimal condition, that is to say, an optimal radius, taking into account also feasible lengths for the design.
The following formulas, extracted by [3], were used, by applying Linear Sloshing Theory:

$$\omega_{mn}^2 = \frac{g\epsilon_{mn}}{R}\tanh\left(\frac{\epsilon_{mn}h}{R}\right) . \tag{5.1}$$

The first mode ($m = 1$) was picked as suggested in [3] for shapes in a rocking motion perpendicular to the axis.

Roots of the Bessel function consist of the values of Eq. ϵ_{mn} 5.1 evaluated for $m = 1$.

The behavior of the tank for an array of radii of:
[3.25, 3.50,3.75, 4.00, 4.25, 4.50,4.75, 5.00] cm was studied.
The corresponding cylinder tank heights are:
[4.22, 3.64, 3.17, 2.78, 2.47, 2.2, 1.98, 1.78] cm.
In Figure 5.2 it is displayed the natural frequency of the sloshing natural mode for $m = 1$ and $n = 1$; it can be seen that natural frequency decreases with increasing radius.

Figures 5.3-5.6 show trends of Inertia Force F_x and Inertia Moment M_y for the established radius values for a harmonic translation motion along x and Pitching motion about y.

The main considerations that may be extracted:

- Inertia Forces decrease with increasing radius
- Pitching moment on the walls decreases with increasing radius

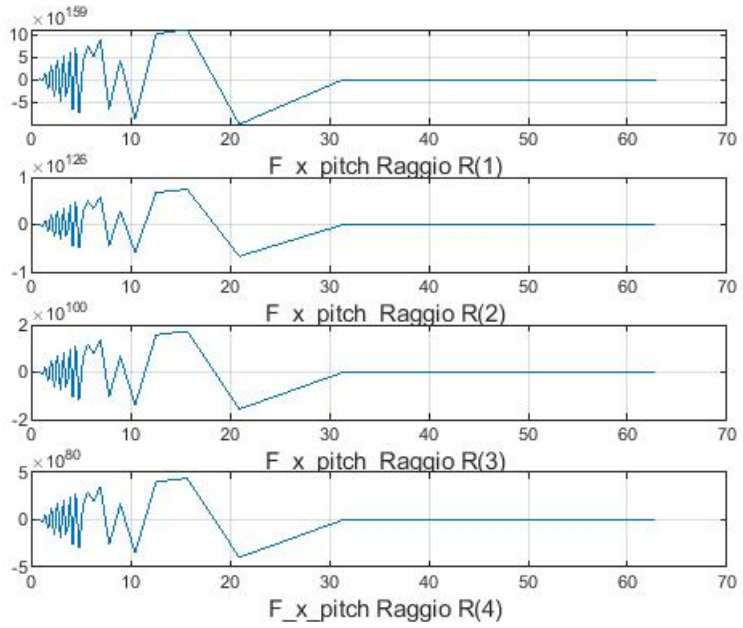


Figure 5.3 Hydrodynamic sloshing force along x versus radio for harmonic motion

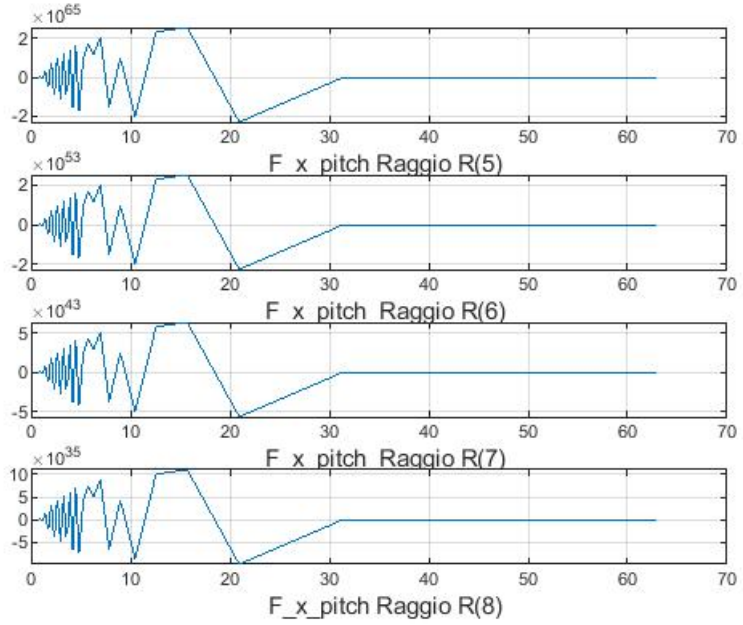


Figure 5.4 Hydrodynamic sloshing force along x versus radio for harmonic motion

- Pitching moment on the bottom decrease with increasing radius (decreasing height)

All this considered a reference dimension of $D = 8\,\mathrm{cm}$ was established for the cylinder as a compromise to guarantee high natural frequencies as well as a moderate bottom pitching moment.

5.1.3 Choice of reference shape

The dimensions obtained were used as a first-step iteration of the design process. The second step consisted in changing the shape of the tank starting from the cylindrical one.
The main drivers of the shape redesign were:

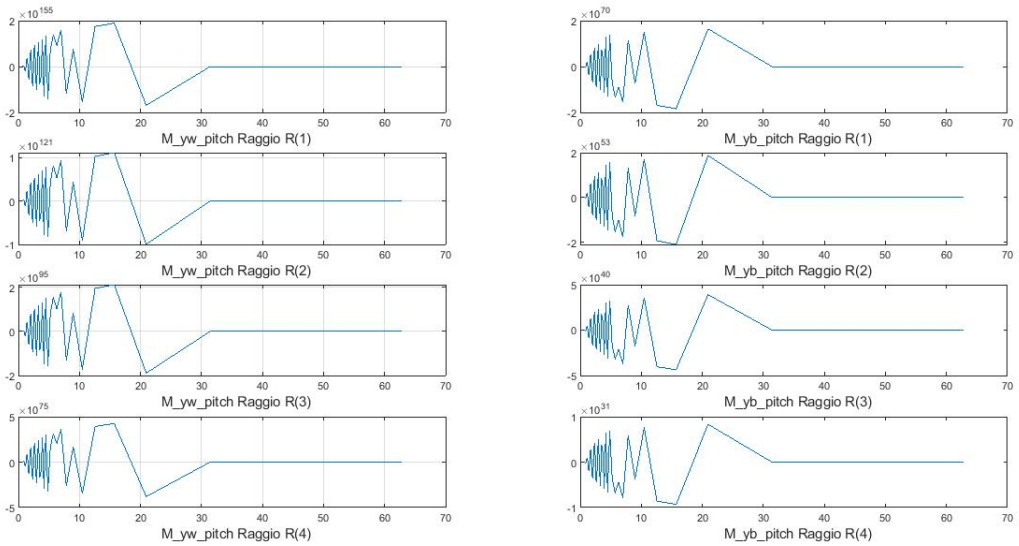


Figure 5.5 On the left there is the Moment of the hydrodynamic forces acting on the tank walls about the y-axis, on the right there is the Moment of the hydrodynamic forces acting on the tank bottom about the y-axis versus radio for harmonic motion

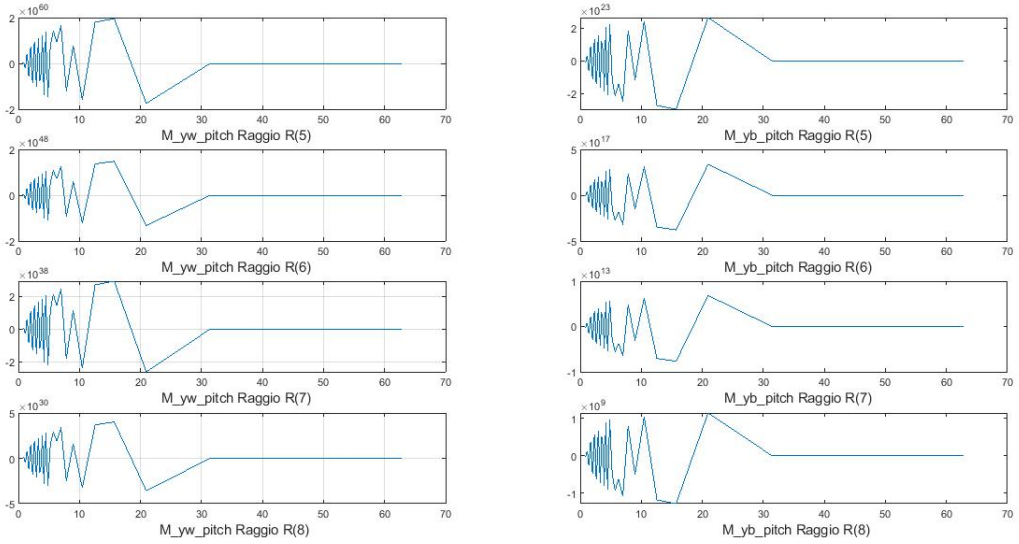


Figure 5.6 On the left there is the Moment of the hydrodynamic forces acting on the tank walls about the y-axis, on the right there is the Moment of the hydrodynamic forces acting on the tank bottom about the y-axis versus radio for harmonic motion

- Creating a confined area where water could accumulate during and after the launch maneuver
- Creating a curved-line profile allowing the sloshing water to be "escorted" as much as possible, dissipating as kinetic energy while not breaking up against sloshing tank walls

The configuration consists of a non-symmetric hourglass comprising a "basin" at the far extreme where it is desired that water can be confined as much as possible.

This shape allows an accumulation of a high layer of water at the frontal tip.

Moreover, this configuration allows a fore location of the center of gravity that enhances the stability features of the configuration.

However, it should be noted that excessive forward location of the center of gravity proved to be detrimental to the gliding phase duration and range performances as pitches down the plane(Chapter 7).

This issue was solved by placing the sloshing mass tank in a more aft location.
2D simulations were performed; flow visualization displayed that reducing the depth of the basin helps to decrease pressure exerted by water on the bottom.

Figures 5.7, 5.8, 5.9, 5.10, 5.11, 5.12 and 5.13 compare a rectangular (lateral "section" of the same dimensions as the cylinder starting configuration) tank simulation with the non-symmetric-hourglass shape figures 5.14, 5.15, 5.16, 5.17, 5.18, 5.19 and 5.20, both excited with a harmonic acceleration.

It can be seen that the developed tank shape helps to keep water in a more advanced position.

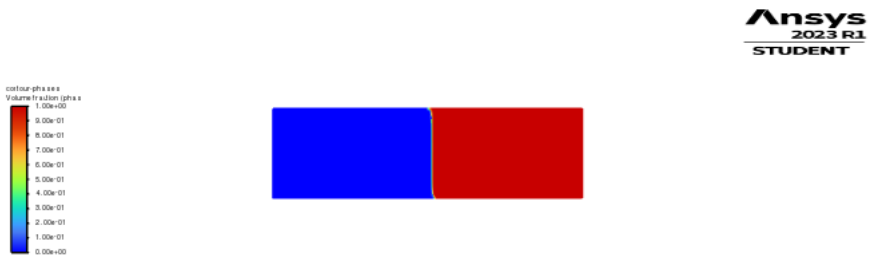


Figure 5.7 Rectangular tank simulation: instant 1

5.1.4 Moving tank simulation and baffles configuration

Once the shape was set, research [3] and [4] allowed us to come up with a starting configuration involving simple baffles arranged along the tank shape.

Iterations were based on outputs related to flow visualizations extracted by simulating a certain configuration in a "launch" condition and a "gliding" condition, where:

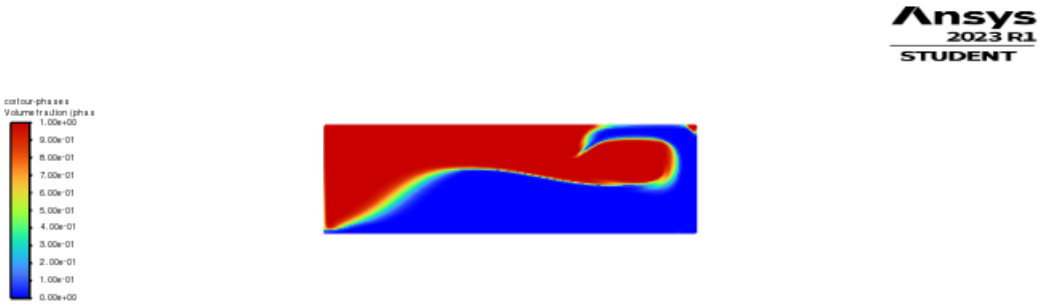


Figure 5.8 Rectangular tank simulation: instant 2

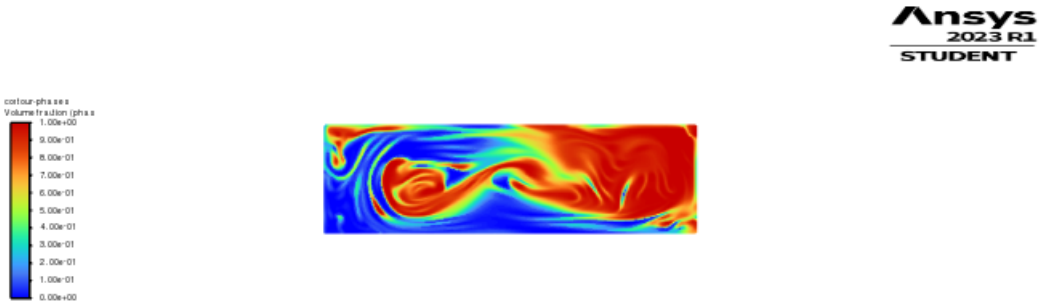


Figure 5.9 Rectangular tank simulation: instant 3

- launch condition consists of a vertical acceleration a_x = 800/5 - (800/5 + 9.81)/0.5 * t for $t \leq 0.5$ and a_x = -9.81 for $t \geq 0.5$
- gliding condition consists of a longitudinal harmonic acceleration added to longitudinal and vertical acceleration due to gravity in a descending trajectory (25 deg)

Proposed configurations are shown in Figures 5.21, 5.22, 5.23, 5.24.
As previously discussed, the main drivers were:

- accumulating as much water as possible at the tip location of the plane
- breaking up coherent motion of water
- not enclosing water too much since it would lead to excessive inertia exertion on structure when gravity and/or further accelerations would lead to water motion

Configuration 1 was intended to create separated regions of "static" flow but proved to be unfavorable since it would prevent the flow from accessing outer regions at launch.
Configuration 2 and 3 would tilt the baffles to allow flow passage towards the outer basin at launch but showed some limitations since flow reaching the outer region would be stuck there and could lead to excessive pressuring of baffles when the fluid would have been set in motion due to flight dynamics.

Among the others, Configuration 4 optimizes sloshing reduction and loads absorption thanks to not-too-high baffles and curved profiles leading to a redirection and natural dissipation of flow dynamics.

Indeed beyond water motion prevention, redirection of flow along with the break up of coherent flow structures was thought to be safer for baffles integrity and inertia loads reduction.

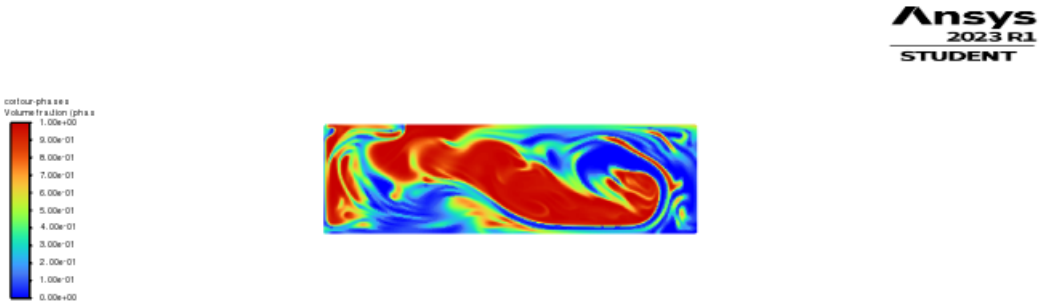


Figure 5.10 Rectangular tank simulation: instant 4

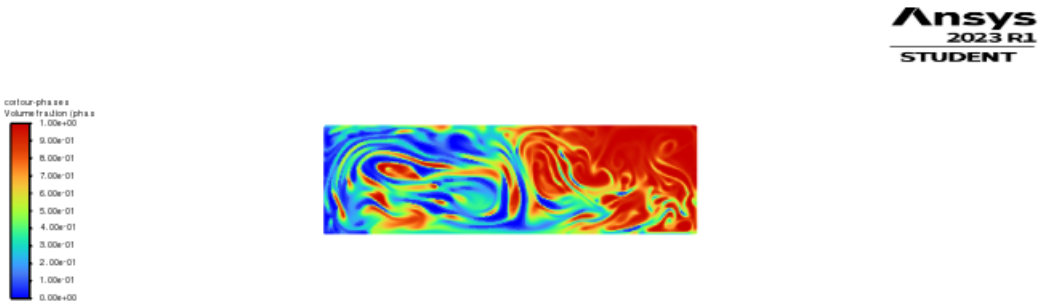


Figure 5.11 Rectangular tank simulation: instant 5

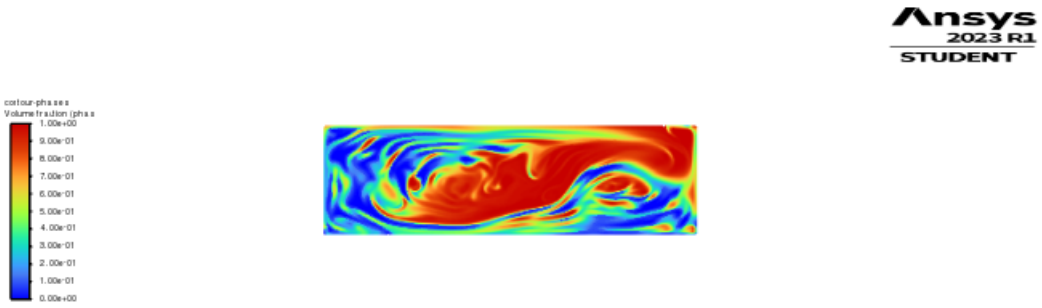


Figure 5.12 Rectangular tank simulation: instant 6

Figures 5.30, 5.31, 5.32, 5.33, 5.34 and 5.35, show the behavior of the Configuration 4 when excited with the mentioned harmonic motion, whereas Figures 5.25,5.26, 5.27, 5.28 and 5.29, display the launch acceleration.

What can be taken away is the fact that flow tends to remain in a fore location despite harmonic motion. Moreover, due to high accelerations at launch, the flow moves at forward locations and is not stopped by physical obstacles that might prevent the flow from reaching those locations in the first place.

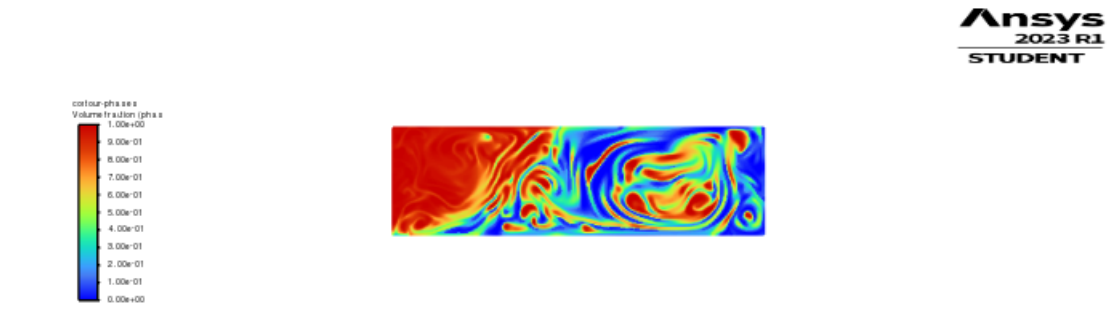


Figure 5.13 Rectangular tank simulation: instant 7

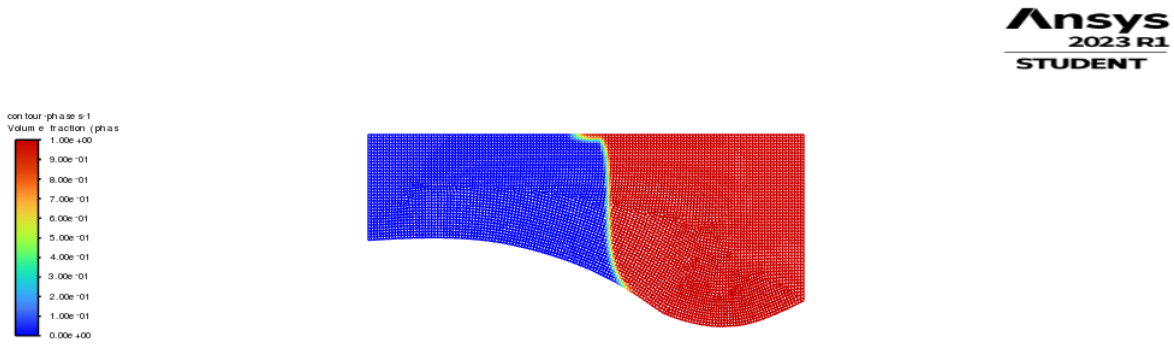


Figure 5.14 Non-symmetric-hourglass shape simulation: instant 1

5.2 Final considerations and tank integration

Configuration 4 was judged as a good convergence point.

The shape can be seen in figure 5.37 and is obtained by solid revolution (180 deg) of the configuration 4 planar shape, which is sized in figure 5.36. The total volume is computed to be 1.41 L.

As can be seen, the roof is flat to match the shape of the fuselage. With regards to the integration of the designed tank in the rocket/glider model, the final configuration is character-

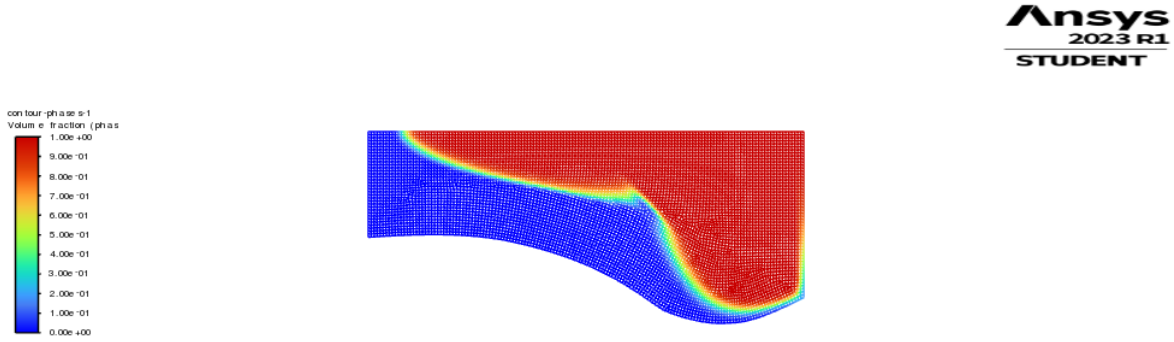


Figure 5.15 Non-symmetric-hourglass shape simulation: instant 2

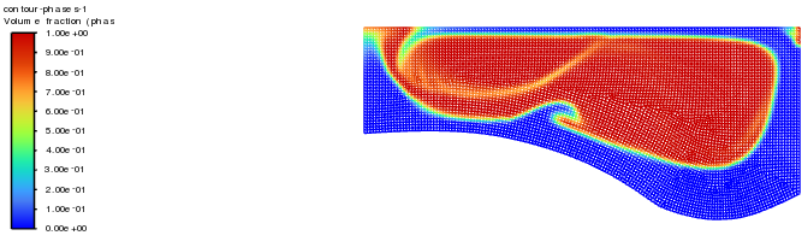


Figure 5.16 Non-symmetric-hourglass shape simulation: instant 3

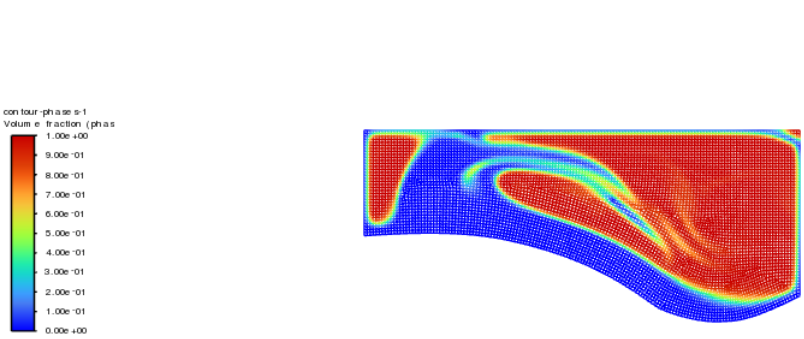


Figure 5.17 Non-symmetric-hourglass shape simulation: instant 4

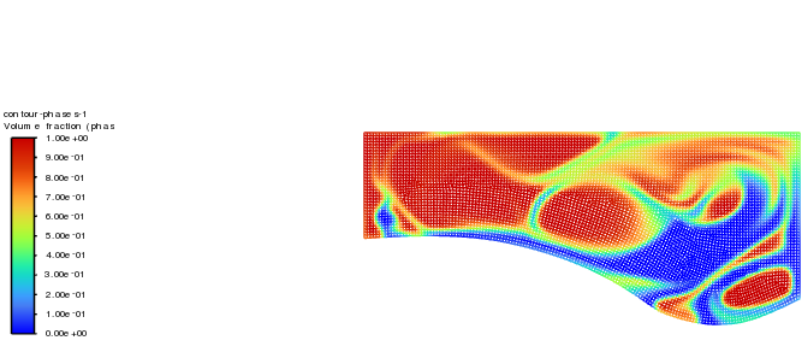


Figure 5.18 Non-symmetric-hourglass shape simulation: instant 5

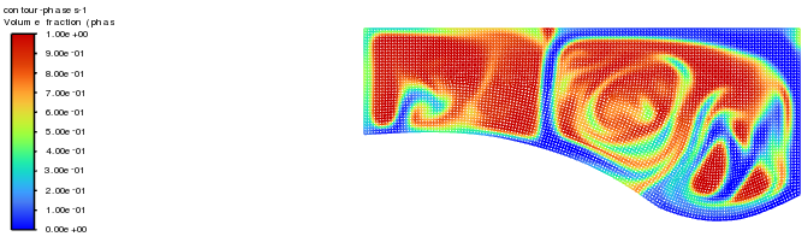


Figure 5.19 Non-symmetric-hourglass shape simulation: instant 6

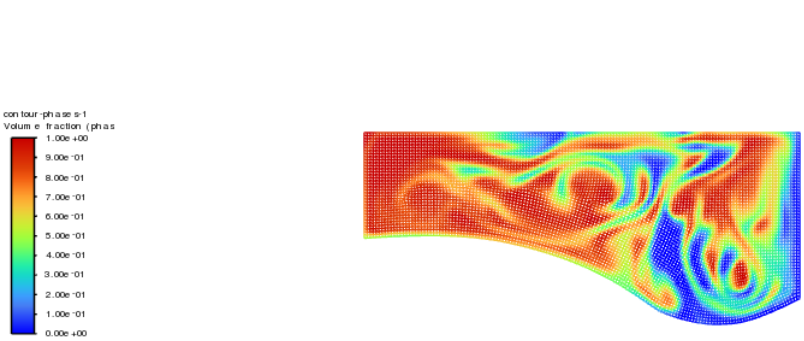


Figure 5.20 Non-symmetric-hourglass shape simulation: instant 7

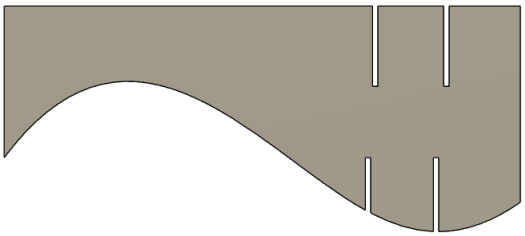


Figure 5.21 Configuration 1

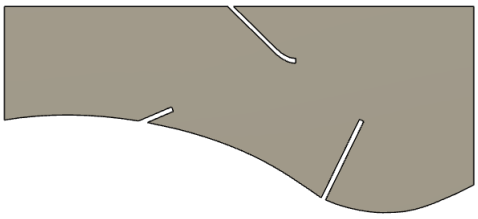


Figure 5.22 Configuration 2

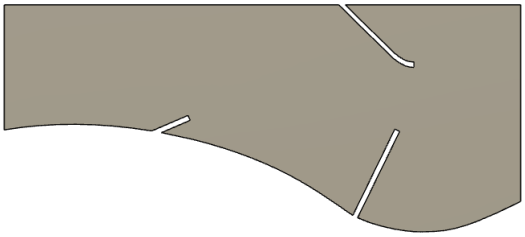


Figure 5.23 Configuration 3

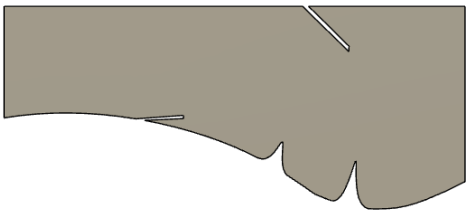


Figure 5.24 Configuration 4

ized by the wider lateral section towards the fore part of the model, placed 20 *cm* from the fore tip.

This choice is justified in Chapter 7 due to some redesign led by the high pitching-down moment of the sloshing water.

While it is true that such a shift of the tank results in a shift of the center of gravity and of the arm of the download of the tail, it also leads to a more impacting reduction of the arm of the sloshing mass.



Figure 5.25 Configuration 4 at launch acceleration: instant 1



Figure 5.26 Configuration 4 at launch acceleration: instant 2



Figure 5.27 Configuration 4 at launch acceleration: instant 3



Figure 5.28 Configuration 4 at launch acceleration; instant 4



Figure 5.29 Configuration 4 at launch acceleration; instant 5



Figure 5.30 Configuration 4, in harmonic motion: instant 1



Figure 5.31 Configuration 4, in harmonic motion: instant 2

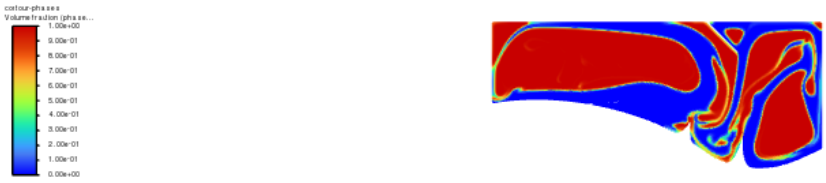


Figure 5.32 Configuration 4, in harmonic motion: instant 3



Figure 5.33 Configuration 4, in harmonic motion: instant 4

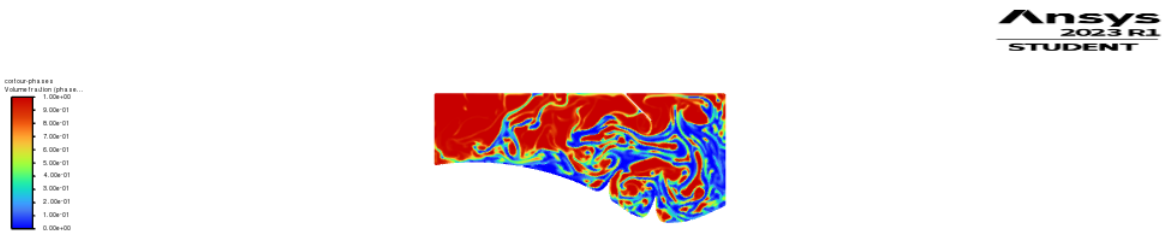


Figure 5.34 Configuration 4, in harmonic motion: instant 5

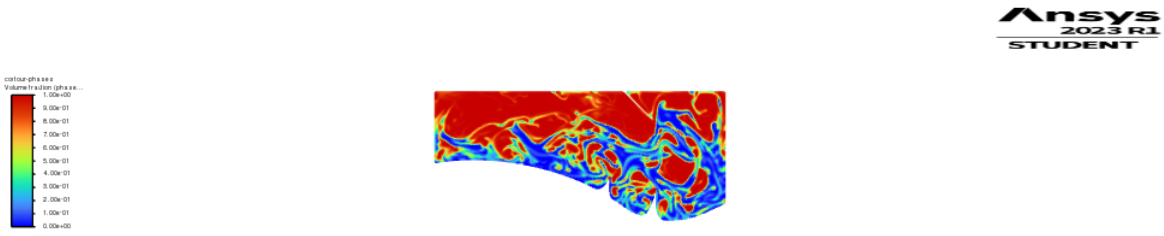


Figure 5.35 Configuration 4, in harmonic motion: instant 6

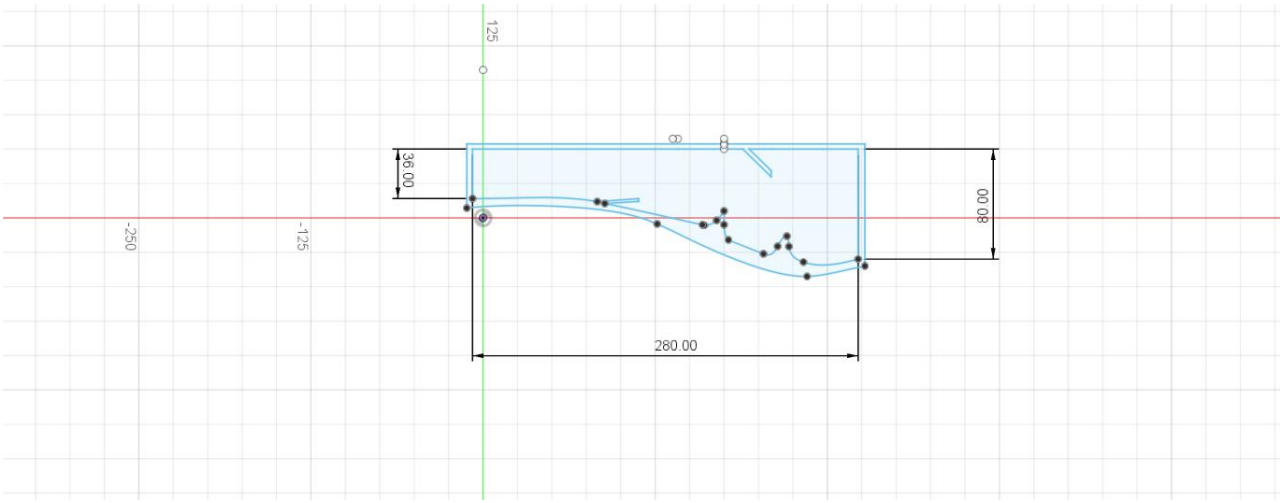


Figure 5.36 Sketch of the section revolution of configuration 4 (Dimension in mm)

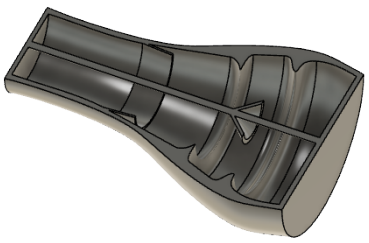


Figure 5.37 Tank complete design

Part III

Test & experiments

Chapter 6

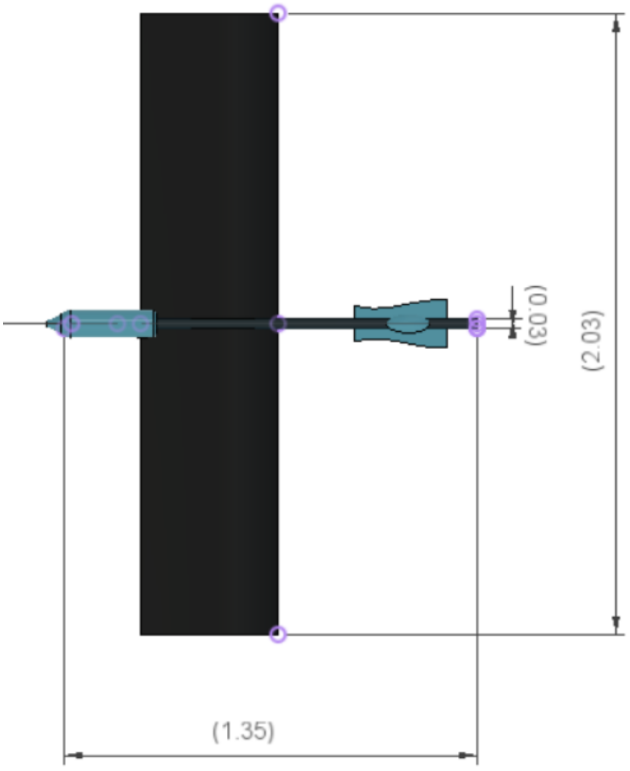
Model validation and Safety assessment

6.1 Model validation

	Requirement	Assestment
Weight	Maximum take-off weight <5 kg (including liquids)	4,1 Kg
Dimensions	Maximum height 1.5 Maximum wingspan 2.25	1.35 m 2.03 m
PropellantTank	Tank Pressure:<10 atm = 147 psi Propellant: water Total mass of propellant less then the mass of sloshing liquid	yes yes yes
Sloshing tank	1 unpressurized sloshing tank containing a minimum of 500 ml and filled half-way Baffles cannot cover more than50% of its cross-sectionalarea Sloshing tank designed to dampen fluid motion	 yes yes yes
Launch and flight	Rocket must be launched vertically (launch angle: 85 degrees) Controlled rocket descent Re-launch within 5 min No parachutes allowed	 yes yes yes yes
Costs	Cost of off-the-shelf parts<300 euro Total cost<500euro	yes yes

Figure 6.1

6.2 Final Configuration



Fusolage	
Height	1.35 m
External Width	0.03 m
Wing	
Wingspan	2.03 m
Wing chord	0.45 m
Position L. e.	0.65 m
Tailplane	
Wing Semi-span	0.35 m
Wing chord	0.15 m
Position L. e.	1.17 m

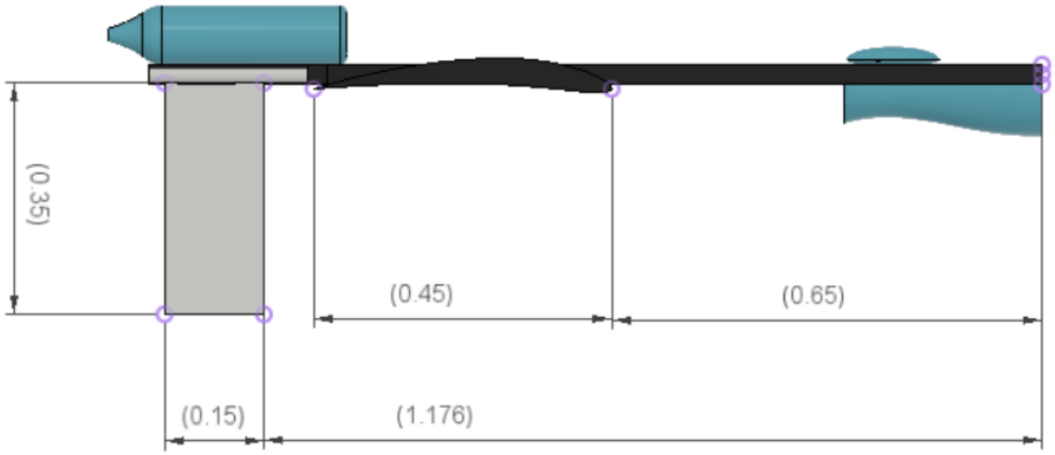


Figure 6.2

6.3 Safety assessment

The hazards that were spotted are mainly related to some stages of the construction.

Identified Hazards	Likelihood Level	Severity Level	Risk Lever (Severity x Likelyhood)	Mitigations
Launch system	2	2	4	safety valve
Wings manufactoring	3	2	6	safety gloves
Fuselage manufactoring	2	3	6	safety glasses
Hotwire	2	2	4	safety gloves
Eletronics manufactoring	1	3	3	eletric tape

Figure 6.3 Risk Assesstment

Figure 6.3 highlights the mentioned events and their estimated frequency.

Chapter 7

Launch tests

7.1 First day



Figure 7.1 First Launch

Extracted information

Regarding experimental data recorded on the first launch day, the following considerations were extracted by testing:

- Wing's structure was verified to be stiff enough to resist impulsive loads at launch.
- Liquid sloshing mass (initially placed at a very fore location) generated a strong pitching moment reducing potentially available range performances.
- Tail was not able to move as predicted.
- Launch tube was too flexible, allowing the prototype to start with a flight-path angle not abiding by requirements.

Some features were changed consequently.

Data acquisition

The following data plots let the reader to better visualize the first flight test history. Figure 7.2 clearly shows the error in angle calculation: the pitch angle never reaches 0 after the lift-off.

Figure 7.3 shows on the left the altitude profile from the turn-on of the system till the end of the flight. On the right, there is a zoom-in of the latest recorded data to better visualize the altitude reached during the flight. From this, follows that the maximum altitude is about $7.5\,\mathrm{m}$.

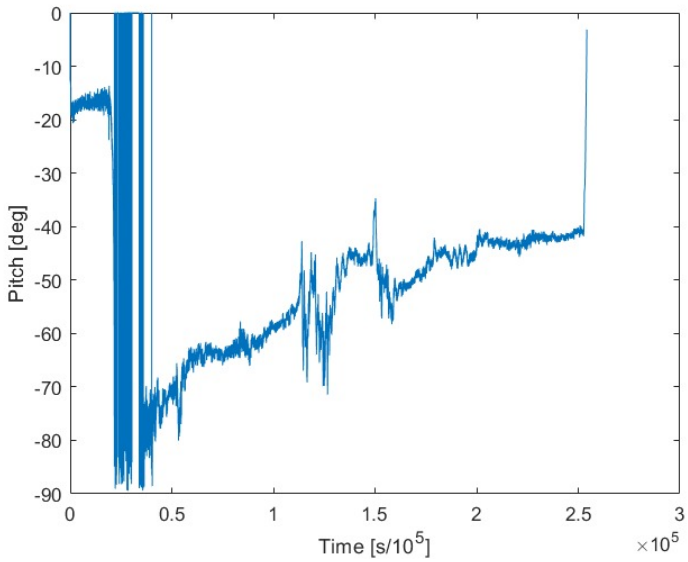
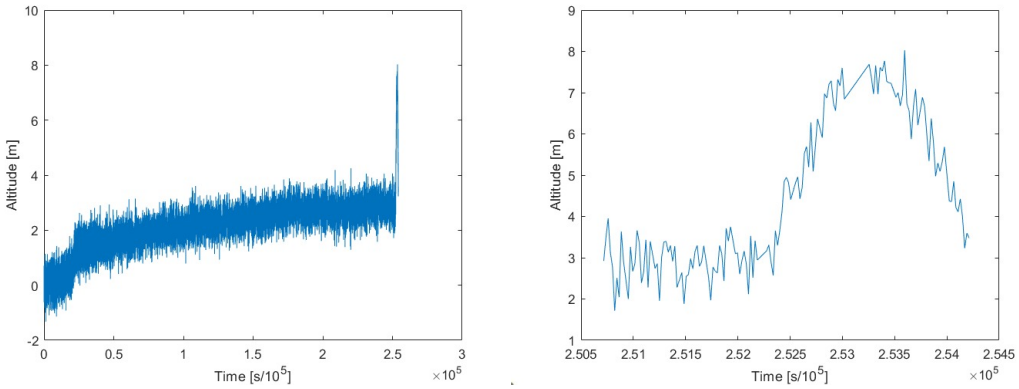


Figure 7.2 Pitch angle time history



(a) *Altitude profile* **(b)** *Zoom-in Altitude profile last 4 seconds*

Figure 7.3 Altitude trend First Launch

Redesign

Accordingly, to experimental results, it was decided that the second launch day would have been carried out with a slightly different configuration:

- Code for the opening of the horizontal tail was varied (see section 3.4.3).
- Sloshing mass was shifted 15 cm rearwards, leading to a significant reduction of sloshing weight force arm at the price of a lower arm of tail aerodynamic force.
- Hardware was shielded better to account for landing.
- Aluminum profiled screwed to launch base was used instead of PVC tube.

7.2 Second day

Extracted information

The second day of launch tests was characterized by two test flights, Launch 2 and Launch 3.
From tests, the following information was extracted:



Figure 7.4 Second Launch



Figure 7.5 Third Launch

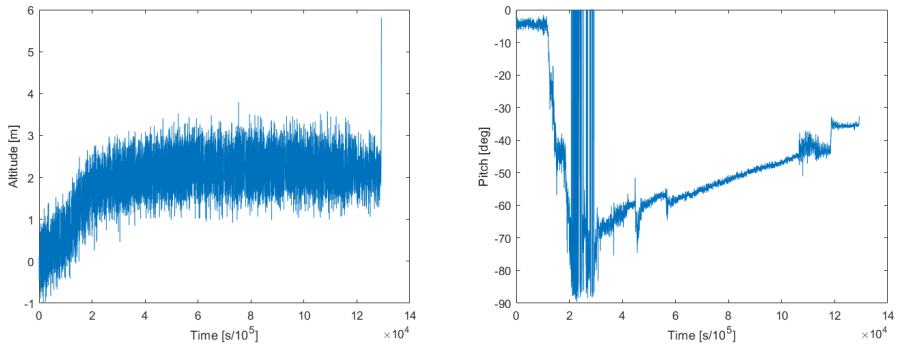
- $\mathcal{R} = 10.5 \div 11\,\mathrm{m}$;
- improvements in tail opening performances;
- still high pitching moment due to sloshing;
- The rocket model was ready to be re-used within 5 minutes.

At the following link, recordings of the tests are provided.

https://youtu.be/y8TNEQP7Qss.

Data acquisition

Unfortunately, regarding the Second Launch, not all the data were saved on the SD. Hence, it was impossible to do further consideration on it. Instead, the Third Launch registered almost all the data, except in the last $2 \div 3$ seconds, due to the break of some wires.
Figure 7.6 shows the trend of the altitude reached and the pitch angle.



(a) *Altitude profile* **(b)** *Pitch angle time history*

Figure 7.6 Data acquisition Third Launch

Final considerations and redesign

The main concerns related to the lack of the tail-induced pitching-up, leading to a lower range and flight time. To address this issue, a redesign was opted for:

- a negative $(-4°)$ tail geometric twist was designed;
- chord = $0.2\,\mathrm{m}$;

- b = $0.5\,\mathrm{m}$;
- sloshing mass was still shifted $5\,\mathrm{cm}$ rearwards.

The redesigned surface's size is calculated to have the same main wing area to horizontal tailplane area ratio of the Airbus A320, just as a reference.

Conclusions

The Airbus Sloshing Rocket Workshop proved to be a unique occasion to sharpen hard skills related to simulation, design, and control; team members pursued their interests and used software that may be relevant to their career or academic perspective.
Furthermore, Team members took this opportunity to grow professionally by respecting deadlines and planning individually and in teamwork.

Although the mission of manufacturing and launching a flying prototype was carried out and the aim of facing all practical problems related to manufacturing and testing was satisfied, the beginning of the design process still relied on a theoretical approach. However, the valuable experience and knowledge taught members to be more aware of bad and good practices in design, meetings, and work management.
Should the experience be repeated, the team intends to come up with a working configuration in a shorter time to go through a longer testing phase.

The project concludes with the development and testing of a functioning and reusable rocket/glider model that abides by the regulations of the Workshop. The research was carried out about the sloshing phenomenon, and the development stage was based on a general understanding of physics and its dependencies.
Future work may consist of further stable design configurations that may be tested. A refinement of the presented model may consist of an even more integrated solution for the sloshing or a solution allowing the control of a larger mass of liquid.

The AirSloths team is fully satisfied with its work, the results that were obtained, and the overall experience. Team members heartily thank AIRBUS, all the speakers, and EUROAVIA for this major opportunity.

CONCLUSIONS

Part IV

Appendix

Chapter A

Appendix

A.1 Sloshing considerations

Liquid sloshing management is an issue that affects a wide range of Engineering applications and leads to various solutions that may or not be applicable in the Aerospace sector. When researching viable options for the Workshop, the design choices may be driven by two aims.

On the one hand, containing the movement of the liquid within the tank by using baffles or other means to restrict the fluid movement may be good to confine the liquid and restrict its movement.

On the contrary, allowing the liquid to move in a tank can help reduce the sloshing forces by providing adequate space for the fluid to slosh and dissipate the energy generated during the motion.

The latter approach is often used in applications where the liquid movement can be accommodated without compromising the integrity or stability of the structure.

About the application here presented, the first solution would allow an efficient center of gravity shift limit. However, in the Concept Design stage, it was supposed that launch accelerations as the rapid attitude variations may lead to excessive loads on baffles, compromising their integrity. Therefore, the team thought of a solution against liquid movement effects on static stability: creating a configuration with very aft-located wings concerning the Center of Gravity to have a certain margin against Center of gravity shifts.

However, the sloshing reduction was realized by inserting some baffles that could redirect the flow and break coherent flow structures but not completely catch and cage the liquid. That should be interpreted as a compromise between the approaches.

A.1.1 Ideas from other fields

To develop sustainable and efficient sloshing control in aviation, several thoughtful ideas and research from other disciplines can inspire. One must bear in mind that applications may vary, and solutions may not be universal.

Structural and Civil Engineering:

Sloshing control can also benefit from advancements in structural engineering. Indeed, innovative materials and designs can minimize vibration and structural deformation.
With that in mind, research may be carried out on vibration-isolation materials that are widely studied in Civil Engineering, such as rubber materials ([5]), porous layers ([6]). Source [7] presents the design, modeling, and prototyping process of a composite tank for cryogenics storage. In [8], damping methods against liquid sloshing is proposed to prevent damage to floating roofs of oil tanks. The paper proposes a buffer using viscoelastic materials as for anti-liquid sloshing measure.

Reference [9] deals with the seismic design of the open cylindrical liquid storage tank to determine convective-dynamic properties (natural frequencies and modes of oscillation), maximum vertical displacements over tank radius, and overall response of the liquid to an earthquake. Opposition to liquid motion is also useful in many environmental safeguards (MOSE, breakwaters).

Control Theory

Advanced control theory and algorithms from fields like robotics, mechatronics, and process control can be adapted to address sloshing challenges. Advanced sloshing control techniques may involve the integration of sensors, actuators, intelligent algorithms, and autonomous control systems.
One way to reduce liquid sloshing is controlling the acceleration or deceleration following the speed of the container([10]). Reference [11] develops a control algorithm to damp the linear lateral sloshing: to keep the upper stage at the desired target attitude, the guidance, navigation, and control (GNC) algorithm commands thruster firings to counter the fluid forces. Further, innovative baffles/tank shape designs may be beneficial for sloshing reduction purposes. In [12], a slot design is proposed for the baffles; in [13], a stepped-roof shape in liquid storage tanks is shown to reduce dynamic sloshing effects. In [14], an automotive-related application tank is simulated, equipped with different kinds of transverse baffles, and subject to constant braking deceleration.

Sloshing proves to be detrimental in liquid storage in space applications, for example by decreasing injector collection efficiency. Classical textbooks ([4], [3]) provide information related to sloshing in microgravity; researches focus on sloshing modeling in microgravity (in [15]) and simulation ([16]).

By incorporating ideas and research from these diverse disciplines, aviation can benefit from interdisciplinary collaboration and innovation to develop sustainable and efficient sloshing control systems.

A.2 Codes

A.2.1 Inertial properties

For the evaluation of the centre of mass position, there is the well-known formula (see [17]):

$$\xi_G = \frac{1}{m} \sum_{i=1}^{n} m_i \xi_i \ ,$$

with $\xi = x, z$. The y coordinate is indeed certainly 0 being the aircraft symmetric.

The `Matlab` code provided in listing A.1 satisfies this request.

Listing A.1

```
function InertiaCell = CalcoloInerzia(m,r)
%% CALCOLOINERZIA Funzione per il calcolo delle proprieta inerziali
% The input are the masses of the material points and their position in the
% adopted reference frame. The matrix r has its rows associated to the material
% points and its columns associated to the specific coordinate x,y,z.
%
% The function output is a cell array which contains the system's:
%%
% * Total mass;
% * Position of the centre of mass;
% * Inertia matrix.
%%
% Dati preliminari
N_el = length(m); % numero di elementi
%%
% Calcolo massa totale
m_tot = 0;
for i = 1:N_el
    m_tot = m_tot + m(i);
end
InertiaCell{1} = m_tot;
%%
% Calcolo posizione del baricentro
r_G = [0;0;0]; % Vettore delle coordiante del baricentro
for j = 1:3
    for i = 1:N_el
        r_G(j) = r_G(j) + m(i)*r(i,j);
    end
    r_G(j) = r_G(j)/m_tot;
end
InertiaCell{2} = r_G;
%%
% Calcolo matrice d'inerzia
I = zeros(3);
for i = 1:N_el
    % Momenti principali
    I(1,1) = I(1,1) + m(i)*r(i,2)^2 + m(i)*r(i,3)^2;
    I(2,2) = I(2,2) + m(i)*r(i,1)^2 + m(i)*r(i,3)^2;
    I(3,3) = I(3,3) + m(i)*r(i,1)^2 + m(i)*r(i,2)^2;
    % Momenti centrifughi
    I(1,2) = I(1,2) + m(i)*r(i,1)*r(i,2);
    I(1,3) = I(1,3) + m(i)*r(i,1)*r(i,3);
    I(2,1) = I(2,1) + m(i)*r(i,2)*r(i,1);
    I(2,3) = I(2,3) + m(i)*r(i,2)*r(i,3);
    I(3,1) = I(3,1) + m(i)*r(i,3)*r(i,1);
    I(3,2) = I(3,2) + m(i)*r(i,3)*r(i,2);
end
InertiaCell{3} = I;
%%
%
end
```

A.2.2 The aerodynamics model

The listing A.2 provides one of the eight `Digital DATCOM+` input files. The code assigns the horizontal tail span in function of the dihedral angle, to correctly describe the same aerodynamic

surface between the different inputs.

These files were changed keeping in mind two priorities:

- to guarantee the aircraft's longitudinal and lateral-directional stabilities;
- to maximize the lift-to-weight ratio.

Listing A.2

```
*
*   Modello DATCOM dell'aliante per la competizione ASRW
*
*   Author : Piercarlo Mirto
*
*   Last update: Aprile 2023.
*
*****************************************************************************
DIM M
DERIV RAD
DAMP
PART

*****************************************************************************
*  Reference Parameters *
*****************************************************************************
 $OPTINS BLREF=1.35, ROUGFC=0.25E-3$



*****************************************************************************
* Group II     Synthesis Parameters
*****************************************************************************
 $SYNTHS SCALE=1.0, VERTUP=.TRUE.,
                 XCG=0.68703, ZCG=-0.0057589,
                 XW=0.65, ZW=0.015,  ALIW=0.0,
                 XH=1.2, ZH=-0.015$




*****************************************************************************
*  Body Configuration Parameters
*****************************************************************************
 $BODY NX=2.0,
       X(1)=0.0, 1.35,
       ZU(1)=0.015, 0.015,
       ZL(1)=-0.015, -0.015,
           R(1)=0.015, 0.015,
           P(1)=0.12, 0.12,
           S(1)=0.0009, 0.0009,
       ITYPE=1.0$



*****************************************************************************
*                      Configurazione dell'ala
*****************************************************************************
 $WGPLNF TYPE=1.0,
                 CHRDR=0.45,   CHRDTP=0.45,
         SSPN=1.0,  SSPNE=0.985,
```

```
         SAVSI=0.0,
         CHSTAT=0.25, TWISTA=0.0,
         DHDADI=0.0$


*  Sezione
NACA W 4 6409!0012



*****************************************************************************
*                       Configurazione del piano orizzontale
*****************************************************************************
 $HTPLNF TYPE=1.0,
                 CHRDR=0.15, CHRDTP=0.15,
         SSPN=0.33807, SSPNE=0.33807,
         SAVSI=0.0,
         CHSTAT=0.25, TWISTA=0.0,
         DHDADI=-15.0$


* Profilo
NACA H 4 0009



*****************************************************************************
*  Flight Conditions *
*****************************************************************************
 $FLTCON WT=5.0, LOOP=2.0,
         NMACH=1.0,
                 MACH(1)=0.005,
         NALT=1.0,
                 ALT(1)=0.0,
         NALPHA=20.0,
         ALSCHD(1)= -16.0, -8.0, -6.0, -4.0, -2.0, 0.0, 2.0, 4.0, 8.0, 9.0,
             10.0, 12.0, 14.0, 16.0, 18.0, 19.0, 20.0, 21.0, 22.0, 24.0,
         STMACH=0.6, TSMACH=1.4, TR=1.0$


CASEID Airbus Glider
```

A.2.3 Complete code

Listing A.3 provides the complete `Matlab` code. The function `eqLongDynamics` contains the Full System of Flight Dynamics Equations.

Listing A.3

```
%% *Competizione ASRW 2023*

clearvars; close all; clc;
%% Codice per il calcolo della posizione del baricentro
% *Authors: Piercarlo Mirto, Vincenzo Junior Di Rosa, Gabriele Salomone.*
%
% Il codice considera il velivolo dopo il rilascio dell'acqua. Dati i tempi
% brevissimi di svuotamento, possiamo dare per assodato che il sistema di punti
% materiali sia fissato per tutta la missione.
```

```
%%
%% Assegnazione propriet  dei materiali

% <> [kg/m^3]
Density.Balsa.Weak = 50;
Density.Balsa.Strong = 160;

Density.Fibra = 2100;

Density.Plywood.Pioppo = 445;
Density.Plywood.Betulla = 635;

Density.Polystyrene.XPS = 35;
Density.Polystyrene.EPS = 25;

% kg/mL^3
Density.Water = 0.997e-3;
%% Propriet  della struttura

N_el = 11;

% Tensore delle coordinate dei baricentri parziali
r = zeros(N_el,3);
m = zeros(1,N_el);
%% Definizione del sistema di punti materiali
% Siccome il velivolo    simmetrico, ci aspettiamo che la posizione y del baricentro
% sia nulla. Siccome per le simulazioni serve anche la matrice d'inerzia, si considerano
% comunque tutte e tre le coordinate delle posizioni dei punti materiali.

% Fusoliera
Fuselage.Length = 1.35;
% <> [m]
Fuselage.ExternalWidth = 3e-2;
Fuselage.SpessoreSezione = 4e-3;
Fuselage.InternalWidth = Fuselage.ExternalWidth - 2*Fuselage.SpessoreSezione;
% <> [m]

Fuselage.Perimeter = 4*Fuselage.ExternalWidth;
Fuselage.Section = Fuselage.ExternalWidth^2;

Fuselage.ExternalVolume = Fuselage.Length * Fuselage.ExternalWidth^2;
Fuselage.InternalVolume = Fuselage.Length * Fuselage.InternalWidth^2;
Fuselage.Volume = Fuselage.ExternalVolume - Fuselage.InternalVolume;
% <> [m^3]

m(1) = Fuselage.Volume * Density.Plywood.Pioppo;
r(1,:) = [Fuselage.Length/2;
          0;
          0];

%%
%
%% Parametri $SYNTHS ...$

XW = .65;
ZW = Fuselage.ExternalWidth/2; % L'ala viene inserita
% in corrispondenza della sua corda

XH = 1.2;
ZH = -Fuselage.ExternalWidth/2 + 2e-2; % Si considera lo spazio necessario alla rotazione
```

```
%%
%% Altri componenti

% Ali
Wing.Wingspan = 2.0;
Wing.Chord = 0.45;
Wing.Thickness = .09 * Wing.Chord; % Profilo NACA-6409

Wing.Volume = 0.75* ...
              Wing.Wingspan * Wing.Chord * Wing.Thickness;
% Il fattore correttivo    perch  il volume    quello di un parallelepipedo

m(2) = Wing.Volume * Density.Polystyrene.EPS;
r(2,:) = [XW + Wing.Chord/3;
          0;
          ZW];

AerodynamicCentre = [XW + Wing.Chord/4;
                     0;
                     ZW];
%%
%

% Longheroni
LongitudinalSpar.Wingspan = Wing.Wingspan;
LongitudinalSpar.Width_X = 8e-3;
LongitudinalSpar.Width_Y = 8e-3;
LongitudinalSpar.Thickness = 1e-3;

LongitudinalSpar.Volume = LongitudinalSpar.Wingspan * LongitudinalSpar.Width_X *
    LongitudinalSpar.Width_Y ...
                          - ...
                          LongitudinalSpar.Wingspan * (LongitudinalSpar.Width_X -
    LongitudinalSpar.Thickness) * (LongitudinalSpar.Width_Y - LongitudinalSpar.Thickness);
% Il volume    quello di un parallelepipedo cavo

m(3) = LongitudinalSpar.Volume * Density.Fibra;
r(3,:) = [XW + Wing.Chord * 2/10;
          0;
          ZW];

m(4) = LongitudinalSpar.Volume * Density.Fibra;
r(4,:) = [XW + Wing.Chord * 7/10;
          0;
          ZW];
%%
%

% Serbatoio sloshing (bottiglia da 1 litro)
SloshingBottle.Mass = 35e-3;
% <> [kg]

SloshingBottle.Radius = 3.5e-2;
SloshingBottle.Height = 22e-2;

SloshingBottle.WaterMass = 700e-3;
% <> [kg]

m(5) = SloshingBottle.WaterMass + SloshingBottle.Mass;
r(5,:) = [SloshingBottle.Height/2;
```

```
128           0;
129           -Fuselage.ExternalWidth/2 - SloshingBottle.Radius];
130 %%
131 %
132
133 % Bottiglia propulsore
134 Scostamento = 5e-2;
135 % <> [m]
136 Bottle.Radius = 4.75e-2;
137 Bottle.Height = 36e-2;
138
139 Bottle.WaterMass = 700e-3;
140
141 EmptyBottle.Mass = 50e-3;
142 m(6) = EmptyBottle.Mass;
143 r(6,:) = [Fuselage.Length - Bottle.Height/2 + Scostamento;
144           0;
145           Fuselage.ExternalWidth/2 + Bottle.Radius]; % Bottiglia centrale
146 %%
147 %
148
149 AperturaVerticale = 0;
150 % <> [  ]
151 %%
152 %
153
154 % Piano di coda
155 HT.Chord = .15;
156 % <> [m]
157 HT.SemiSpan = .35;
158 HT.Width = .09 * HT.Chord; % NACA 0009
159
160 HT.SemiaperturaInPianta = HT.SemiSpan * cosd(90 - AperturaVerticale);
161
162 HT.HalfVolume = 0.75 * ...
163                 HT.Chord * HT.Width * HT.SemiSpan;
164
165 m(7) = 2 * HT.HalfVolume * Density.Polystyrene.EPS;
166 r(7,:) = [XH + HT.Chord/3;
167           0;
168           -ZH - HT.SemiSpan/2*cosd(AperturaVerticale)];
169 %%
170 %
171
172 % Longherone del piano di coda
173 HTSpar.Length = HT.SemiSpan;
174 HTSpar.Width_X = 8e-3;
175 HTSpar.Width_Y = 8e-3;
176 HTSpar.Thickness = 1e-3;
177
178 HTSpar.Volume = HTSpar.Length * HTSpar.Width_X * HTSpar.Width_Y ...
179                 - ...
180                 HTSpar.Length * (HTSpar.Width_X - HTSpar.Thickness) * (HTSpar.Width_Y -
    HTSpar.Thickness);
181
182
183 m(8) = 2 * HTSpar.Volume * Density.Fibra;
184 r(8,:) = [XH + HT.Chord/2;  % In corrispondenza del baricentro dell'HT
185           0;
186           r(7,3)];
```

```
%%
%

% Motore movimentazione piano di coda
Motor.Mass = 300e-3; % Massa comprensiva dei due motori + il loro supporto
% <> [kg]

m(9) = Motor.Mass;
r(9,:) = [Fuselage.Length + Scostamento - HT.Chord/2;
          0;
          -ZH];
% Si trascura lo spessore del motorino, dell'ordine di mezzo centimetro
% Non parte dalla fine della fusoliera, ma segue la stessa logica del piano
% di coda e delle bottiglie
%%

% Motore movimentazione piano di coda
Arduino.Mass = 60e-3;
% <> [kg]

m(10) = Arduino.Mass;
r(10,:) = [Fuselage.Length - Bottle.Height;
          0;
          Fuselage.ExternalWidth];
%%
%

% Batteria
Batteria.Mass = 122e-3;
% <> [kg]

m(11) = Batteria.Mass;
r(11,:) = [Fuselage.Length - Bottle.Height;
          0;
          Fuselage.ExternalWidth];
%%
%
%% Calcolo propriet  inerziali

A = CalcoloInerzia(m,r);

m_tot = A{1};
m_tot_cons = 1.15 * m_tot;
m_velivolo = m_tot_cons + Bottle.WaterMass;
r_G = A{2}; % [m]
I = A{3}; % Matrice d'inerzia, [kg*m^2]
%%

disp([ ...
    'Valori fissati:', ...
    newline, ...
    '------------------------------------------------------------', ...
    newline, ...
    'Numero bottiglie: 1' ...
    newline, ...
    'Massimo angolo di apertura: ', num2str(AperturaVerticale), ...
    newline, ...
    '------------------------------------------------------------', ...
    newline, ...
    'Lunghezza fusoliera: ', num2str(Fuselage.Length), ...
```

```
    newline, ...
    'Perimetro sezione fusoliera: ', num2str(Fuselage.Perimeter), ...
    newline, ...
    'Area di sezione fusoliera: ', num2str(Fuselage.Section), ...
    newline, ...
    '-------------------------------------------------------------', ...
    newline, ...
    'Apertura alare: ', num2str(Wing.Wingspan), ...
    newline, ...
    'Corda alare: ', num2str(Wing.Chord) ...
    newline, ...
    'Posizione XW apice alare: ',num2str(XW), ...
    newline, ...
    'Posizione ZW apice alare: ',num2str(ZW), ...
    newline, ...
    '-------------------------------------------------------------', ...
    newline, ...
    'Corda piano di coda: ', num2str(HT.Chord) ...
    newline, ...
    'Semiapertura in pianta del piano di coda: ', num2str(HT.SemiaperturaInPianta) ...
    newline, ...
    'Posizione XH apice piano orizzontale: ',num2str(XH), ...
    newline, ...
    'Posizione ZH apice piano orizzontale: ',num2str(ZH), ...
    newline, ...
    '-------------------------------------------------------------', ...
    ]);
disp(['Valori oggetto di iterazione:' ...
    newline, ...
    'Posizione apice ala: ',num2str(XW), ...
    newline, ...
    'Apertura piano di coda: ', num2str(HT.SemiSpan), ...
    newline, ...
    '-------------------------------------------------------------', ...
    newline, ...
    'XG: ',num2str(r_G(1)), ...
    newline, ...
    'ZG: ',num2str(r_G(3))]);
%%
% Questi dati vengono insetiti a mano nel file DATCOM - Rocket Mode
%
% Per il momento si ignora il fatto che in fase di decollo il numero di Mach
% sia superiore a quello in condizione di glider
%% Caricamento dati

global Data myConfig
%%
%

Data.Rock = datcomimport('Airbus_Predator--Rocket_mode.out');
Data.Rock = Data.Rock{1,1};

Data.Glider10 = datcomimport('Airbus_Predator--Glider_mode_10.out');
Data.Glider10 = Data.Glider10{1,1};

Data.Glider20 = datcomimport('Airbus_Predator--Glider_mode_20.out');
Data.Glider20 = Data.Glider20{1,1};

Data.Glider30 = datcomimport('Airbus_Predator--Glider_mode_30.out');
Data.Glider30 = Data.Glider30{1,1};
```

```
307
308 Data.Glider40 = datcomimport('Airbus_Predator--Glider_mode_40.out');
309 Data.Glider40 = Data.Glider40{1,1};
310
311 Data.Glider50 = datcomimport('Airbus_Predator--Glider_mode_50.out');
312 Data.Glider50 = Data.Glider50{1,1};
313
314 Data.Glider60 = datcomimport('Airbus_Predator--Glider_mode_60.out');
315 Data.Glider60 = Data.Glider60{1,1};
316
317 Data.Glider70 = datcomimport('Airbus_Predator--Glider_mode_70.out');
318 Data.Glider70 = Data.Glider70{1,1};
319
320 Data.Glider75 = datcomimport('Airbus_Predator--Glider_mode_75.out');
321 Data.Glider75 = Data.Glider75{1,1};
322 % <> [deg]
323 %% Simulazione del volo dell'aeromodello
324 % NOTA: il BRF ha z verso la basso, mentre DATCOM costruisce il velivolo verso
325 % l'alto
326
327 global rhoW Aout tf ...
328        Preliminary
329 %%
330 %
331
332 rhoW = 997; Aout = 22^2*pi/4*10^-6;
333
334 tf = 10 ;
335 %%
336 %% Costruzione delle classi
337 % Dati universali (sorgente della struttura dati del modello aerodinamico)
338
339 % Geometry
340 Preliminary.b = Wing.Wingspan;
341 Preliminary.mac = Wing.Chord;
342 Preliminary.S = Preliminary.b * Preliminary.mac;
343 Preliminary.AR = Preliminary.b^2/Preliminary.S;
344
345 % Inertia
346 Preliminary.mass = m_tot;
347 Preliminary.W = Preliminary.mass * 9.81;
348 Preliminary.k_y = sqrt(I(2,2)/Preliminary.mass);
349 Preliminary.PosizioneG = r_G;
350 Preliminary.RadiusBottle = Bottle.Radius;
351 Preliminary.FuselageWidth = Fuselage.ExternalWidth;
352 %%
353 % La descrizione completa della struct del velivolo viene fatta implicitamente
354 % dentro |eqLongDynamics.mlx|, che a sua volta la carica da |ModelloAerodinamico.mlx|
355 %
356 %
357 %% Condizioni iniziali
358
359 theta0 = convang(88.00,'deg','rad'); % convang(88.00,'deg','rad');    % Angolo di rampa
360 z0 = 0; xE0 = 0; yE0 = 0;
361
362 [air_Temp0,sound_speed0,air_pressure0,rho0] = atmosisa(-z0);
363
364 V0 = 0.01; q0 = 0;
365 alpha0 = 0;
366
```

```
367
368 x0 = [V0, alpha0, q0, xE0, z0, theta0];
369
370 theta_switch = 40;
371 %%
372 %% Avvio simulazione
373
374 % options = odeset('RelTol', 1e-3,'AbsTol', 1e-3*ones(1,6));
375 [vTime, mState] = ode15s(@eqLongDynamics,[0 tf],x0);
376 mState(:,2) = convang(mState(:,2),'rad','deg');
377 mState(:,3) = convang(mState(:,3),'rad','deg');
378 mState(:,6) = convang(mState(:,6),'rad','deg');
379
380 AndamentoQuota = @(t) ...
381                  interp1(vTime,-mState(:,5),t,'linear');
382 %%
383 %
384
385 % Assegnazione della tolleranza accettata
386 Tol = 1e-3;
387 % Estremi d e l l intervallo  di partenza
388 xs = vTime(2);  fs = AndamentoQuota(xs);
389
390 for i = 2:length(vTime)
391     xd = vTime(i); fd = AndamentoQuota(xd);
392     if (fs*fd < 0)
393         break
394     end
395 end
396 if (fs*fd > 0)
397     error('La funzione non assume mai valore 0');
398 end
399 % Verifica pre-processo
400 if (fs*fd == 0)
401     error('Zero g i   trovato');
402     end
403
404 % Punto medio e relativa immagine
405 xm = (xs+xd)/2; fm = AndamentoQuota(xm);
406
407 it = 0;
408 while abs(fm)>Tol
409     it = it+1;
410     % Processo di restrizione d e l l intervallo . Se i valori di f su due punti sono discordi,
411     % allora quello    l   intervallo   giusto, e l  altro   estremo deve essere aggiornato
412     if (fs*fm<0)
413         xd = xm;
414         fd = fm;
415     else
416         xs = xm;
417         fs = fm;
418     end
419     % Aggiornamento
420     xm = (xs+xd)/2; fm = AndamentoQuota(xm);
421 end
422 t_fin = xm;
423 FlightTime = linspace(0,t_fin,100);
424
425 AndamentoV = @(t) ...
426              interp1(vTime,mState(:,1),t,'linear');
```

```
427 AndamentoX = @(t) ...
428             interp1(vTime,mState(:,4),t,'linear');
429 AndamentoTheta = @(t) ...
430                 interp1(vTime,mState(:,6),t,'linear');
431 AndamentoAlpha = @(t) ...
432                 interp1(vTime,mState(:,2),t,'linear');
433 Andamentoq = @(t) ...
434             interp1(vTime,mState(:,3),t,'linear');
```

A.2.4 Control System

Introduction

In this section, you can find all the code utilized to control the tailplane. To better approach these codes there is a clarification about the angle sign to do. For the correct functioning of the IMU filter, the input angle for the If() condition has the opposite sign that supposes to have. For example, this construct "if (pitch < -10.00)" refers to a positive pitch angle of the real configuration of the vector and you should read it as " if (pitch> 10.00)".

First test

The first version of the code is provided in listing A.4.

Listing A.4

```
/**
 ******************************************************************************
 * @file    main.cpp
 * @author  Pierluca De Felice
 * @version V1.2.2
 * @date    22-April-2023
 * @brief   Unmanned model plane stabilization using the X_NUCLEO_IKS01A3
 *          MEMS Inertial & Environmental Sensor Nucleo expansion board.
 ******************************************************************************
*/

//NOTE: when the MCU is connected to a computer, to print data on screen the "printf" function
//      should be uncommented. Remember to put them in comment again before preparing the MCU for the launch.

/* Includes */
#include "mbed.h"
#include "rtos.h"
#include "XNucleoIKS01A3.h"
#include "IMUfilter.h"
#include "SDBlockDevice.h"
#include "FATFileSystem.h"
#include <cstdio>

/* Gravity at Earth's surface in m/s/s */
#define g0 9.812865328f
/* Std air density */
#define density 1.225f
/* Conversion from rad to deg and from deg to rad */
#define todeg 57.2957795f
#define torad 0.01745329252f
/* Number of samples for ground level pressure */
#define SAMPLES 50


/* Defining SD device for storing data */
SDBlockDevice sd(D11, D12, D13, D10); //MOSI MISO SCK CS
FATFileSystem FSys("FSys");

/* Instantiate the expansion board */
static XNucleoIKS01A3 *mems_expansion_board = XNucleoIKS01A3::instance(D14, D15, D4, D5, A3, D6, A4);
```

```
41
42  /* Retrieve the composing elements of the expansion board */
43  static LPS22HHSensor *press_temp = mems_expansion_board->pt_sensor;
44  static LSM6DSOSensor *acc_gyro = mems_expansion_board->acc_gyro;
45
46  /* Defining the user led as output */
47  DigitalOut led(LED1);
48
49  /* Defining the user button as input */
50  InterruptIn button(BUTTON1);
51
52  /* Define the pwm pin that will send the signal to move the servo */
53  PwmOut tailL(D3);
54  PwmOut tailR(D5);
55
56  /* Filter for angles calculation */
57  IMUfilter imuFilter(0.1, 10);
58
59  /* Defining an int for countng the phase of the mission reached */
60  int phase = 0;
61
62  /* User button callback. When the button is pressed phase is increased by one */
63  void phase_add()
64  {
65      phase++;
66  }
67
68  /* Setting a timer for mesuring time during the filght */
69  Timer tempo;
70  int t = 0;
71  int intervallo = 0;
72
73  /* Simple main function */
74  int main()
75  {
76      /* Attach callback to User button press */
77      button.fall(&phase_add);
78
79      uint8_t id;
80      /* Declaring variables for taking angle mesurements */
81      int16_t Accel[3], Gyro[3];
82      float Accms2[3], Gyrads[3];
83      float fullscale, pitch, roll;
84
85      /* Declaring variables for calibrating the sensor and calculating the altitude during the
         flight */
86      float init_press = 0.0f;
87      float altitude = 0.0f;
88      float pressure = 0.0f;
89      float med_press = 0.0f;
90      float press_buffer = 0.0f;
91
92      /* Setting LSM6DSO */
93      acc_gyro->set_x_odr(208.0f);
94      acc_gyro->set_g_odr(208.0f);
95      acc_gyro->set_x_fs(16.0f);
96      acc_gyro->set_g_fs(1000.0f);
97
98      /* Enable all sensors */
99      press_temp->enable();
```

```
100     acc_gyro->enable_x();
101     acc_gyro->enable_g();
102 
103     /* Setting LPS22HH */
104     press_temp->set_odr(200.0f);
105 
106     /* Mounting the SD device */
107     FSys.mount(&sd);
108 
109     /* Creating a file on the SD device and starting the program */
110     FILE *fd = fopen("/FSys/mbed1.txt", "w");
111 
112     if(fd==NULL) {
113         /* Something went wrong */
114         for(int i=0; i<50; i++){
115         led=!led;
116         ThisThread::sleep_for(3000ms); // NOTE: If the led turns on and off for 3s, something
     has gone wrong with the sd. The program will run anyway but data will not be stored.
117         }
118     }
119     else {
120         fprintf(fd, "Welcome SD!!\r\n");
121 
122         //printf("It works!\r\n");
123     }
124 
125 
126     //printf("\r\n--- Starting new run ---\r\n");
127 
128     /* Cheking if the sensors work properly */
129     press_temp->read_id(&id);
130     //printf( "LPS22HH  pressure & temperature   = 0x%X\r\n", id);
131     fprintf(fd, "LPS22HH  pressure & temperature   = 0x%X\r\n", id);
132 
133     acc_gyro->read_id(&id);
134     //printf( "LSM6DSO accelerometer & gyroscope = 0x%X\r\n", id);
135     fprintf(fd, "LSM6DSO accelerometer & gyroscope = 0x%X\r\n", id);
136 
137     acc_gyro->get_x_fs(&fullscale);
138     //fprintf(fd, "%f \r\n", fullscale);
139     printf( "%f \r\n", fullscale);
140 
141     acc_gyro->get_g_fs(&fullscale);
142     //fprintf(fd, "%f \r\n", fullscale);
143     printf( "%f \r\n", fullscale);
144 
145 
146     /* Getting multiple mesurements with the LPS22HH pressure sensor to set the ground level
     */
147     for( int i=0; i<50; i++){
148 
149         press_temp->get_pressure(&pressure);
150         pressure = pressure*100;
151         press_buffer = press_buffer + pressure;
152         //printf(" %d ", i);
153         ThisThread::sleep_for(5ms);
154 
155     }
156     init_press = press_buffer / 50;
157     press_buffer=0.0f;
```

```
158
159
160     /* Printing initial data on the SD Card */
161     //printf("Ground Level Pressure: %f \r\n", init_press);
162     fprintf(fd, "Ground Level Pressure: %f \r\n", init_press);
163     fprintf(fd, "Pitch (deg) \t\t Roll (deg)\t\t Alt (m)\t\t Time (ms)\r\n");
164
165     /* Setting the servo parameters */
166     tailL.period_ms(20);
167     tailL.pulsewidth_us(1500);
168     tailR.period_ms(20);
169     tailR.pulsewidth_us(2500);
170
171     /* Starting the timer */
172     tempo.start();
173     tempo.reset();
174     while (1) {
175
176         /* Getting values from the sensor */
177         acc_gyro->get_x_axes_raw(Accel);
178         acc_gyro->get_g_axes_raw(Gyro);
179         t = tempo.read_ms();
180
181         /* Converting the values from the LSM6DSO into mesures to be used in the filter (rad/
    s and m/s/s) */
182         for(int i=0; i<3; i++){
183
184             Accms2[i]=Accel[i];
185             Accms2[i]=(Accms2[i]*g0)/1000;
186             Gyrads[i]=Gyro[i];
187             Gyrads[i]=(Gyrads[i]*torad)/1000;
188             }
189
190         /* NOTE: Gyro and accelerometer values must be converted to rad/s and m/s/s
191             before being passed trough the filter. */
192         imuFilter.updateFilter(Gyrads[0],
193                                Gyrads[1],
194                                Gyrads[2],
195                                Accms2[0],
196                                Accms2[1],
197                                Accms2[2]);
198         imuFilter.computeEuler();
199
200         pitch = imuFilter.getPitch()*todeg; //to degrees
201         roll = imuFilter.getRoll()*todeg; //to degrees
202
203         //printf("pitch = %f \r\n", pitch);
204
205         /* Calculating the altitude from pressure data with the Hydrostatic Equation*/
206         press_temp->get_pressure(&med_press);
207         altitude = - ((med_press*100 - init_press) / (1.225f * 9.812865328f)) ;
208         fprintf(fd, "%f \t %f \t %f \t %d \r\n", pitch, roll, altitude, t);
209
210
211
212         /* The variable "phase" indicates the mission phase. To manually advance the phase
    press the User Button.
213             To get from phase 0 to phase 1 is necessary to press the User Button once.
214             To get from phase 1 to phase 2 the aircraft should change its orientation from
    vertical to the specified angle.
```

```
215         To get from phase 2 to phase 3 is necessary to press the User Button once.
216       NOTE: The Launch Phase (case 1) should only start when the aircraft is mounted on the launchpad! Remember to press
217             the User Button for the first time only after placing the aircraft in the desired position. */
218
219     switch(phase){
220
221         case 1: // Launch Phase: the aircraft will wait to reach the angle specified in the if() to start the Gliding Phase
222
223         if(pitch<-10.00){
224             //printf("Still vertical\r\n");
225         } else {
226             //printf("Starting glider phase\r\n");
227             phase++;
228             /* Setting the stable tailplane position. */
229             // tailL.pulsewidth_us(1800);
230             // tailR.pulsewidth_us(2200);
231         }
232
233         break;
234
235         case 2: // Gliding Phase: the aircraft changes the opening angle of the tailplane in order to keep a stable
236                 //                position at different pitch angles.
237
238         ThisThread::sleep_for(19ms);
239
240         if (pitch>1.00 && pitch <5.00)
241                 {//printf("stable \r\n");
242                  tailL.pulsewidth_us(2300);
243                  tailR.pulsewidth_us(1700);
244                  }
245         else if (pitch>5.00 && pitch<=10.00)
246                 {//printf("too low... \r\n  correcting... \r\n");
247                  tailL.pulsewidth_us(2325);
248                  tailR.pulsewidth_us(1675);
249                  }
250         else if (pitch>10.00 && pitch<=15.00)
251                 {//printf("too low... \r\n  correcting... \r\n");
252                  tailL.pulsewidth_us(2350);
253                  tailR.pulsewidth_us(1650);
254                  }
255         else if (pitch>15.00 && pitch<=20.00)
256                 {//printf("too low... \r\n  correcting... \r\n");
257                  tailL.pulsewidth_us(2375);
258                  tailR.pulsewidth_us(1625);
259                  }
260
261         else if (pitch>20.00)
262                 {//printf("too high... \r\n  correcting... \r\n");
263                  tailL.pulsewidth_us(2375);
264                  tailR.pulsewidth_us(1625);
265                  }
266
267         else if (pitch<1)
268                 {//printf("too high... \r\n  correcting... \r\n");
269                  tailL.pulsewidth_us(1500);
270                  tailR.pulsewidth_us(2500);
```

```
                    }

            break;

            case 3: //Closing Phase: the aircraft has landed and the user operates the User
    Button to close the file on the SD Card

            fclose(fd);
            //printf("closing...\r\n");

            while(phase>=3){
                led=!led;
                ThisThread::sleep_for(500ms);
                //printf("closed\r\n");
            }

            break;

            }


    }
}
```

Firs test: data acquisition

Listing A.5

```
Welcome SD!!
LPS22HH  pressure & temperature   = 0xB3
LSM6DSO accelerometer & gyroscope = 0x6C
Ground Level Pressure: 99965.148438
Pitch (deg)      Roll (deg)      Alt (m)      Time (ms)
0.000000         0.000000        0.711659        1

[waiting to the launch]
-0.000000        136.568924      1.284886        26603
-0.000000        164.960281      1.101610        26606
-0.000000        69.799225       1.272538        26634
-0.000000        56.506947       1.256290        26637
-0.000000        52.708195       1.256290        26640
-0.000000        48.854305       0.922883        26643
-0.000000        46.672672       0.922883        26646
-0.000000        35.717678       1.589048        26649
-0.000000        21.167507       1.589048        26652
-0.000000        34.212429       1.122407        26655
-0.000000        19.892639       1.554602        26658
-0.000000        6.836411        1.554602        26661
-0.000000        4.829441        1.826918        26663
-0.000000        13.319563       1.826918        26666
-0.000000        24.151365       1.727480        26695
-0.000000        18.387217       1.727480        26697
-0.000000        9.262537        1.592947        26700
-0.000000        3.194693        1.592947        26703
-0.000000        -1.850380       1.644291        26706
-0.000000        3.606990        1.644291        26709
-0.000000        1.310859        1.028819        26712
-0.000000        -6.115351       1.028819        26715
```

```
-0.000000       4.192911        1.112008        26718
-0.000000       8.209339        1.798321        26721
-0.000000       11.338125       1.798321        26723
-0.000000       16.541426       1.006072        26726
-0.000000       14.096617       1.438917        26754
-0.000000       10.941253       1.138005        26757
-0.000000       0.386970        1.138005        26760
-0.000000       10.943750       1.276438        26763
-0.000000       10.604945       1.276438        26766
-0.000000       23.468805       1.321282        26769
-0.000000       10.518615       1.321282        26772
-0.000000       24.297327       1.953001        26775
-0.000000       13.019586       0.644718        26778
-0.000000       22.481220       0.644718        26780
-0.000000       29.595592       1.577349        26783
-0.000000       24.166739       1.577349        26786
-0.000000       18.918724       1.581249        26814
-0.000000       22.987984       1.581249        26817
-0.000000       19.027340       1.205597        26820
-0.000000       27.435026       1.205597        26823
-0.000000       19.591803       1.904258        26826
-0.000000       30.098288       1.904258        26829
-0.000000       36.367046       1.538354        26832
-0.000000       40.761490       1.339480        26835
-0.000000       43.627502       1.339480        26838
-0.000000       47.909977       1.205597        26841
-0.000000       28.126619       1.205597        26844
-0.000000       32.170258       1.577349        26846
-0.000000       13.632502       1.144504        26875
-0.000000       16.217335       0.927432        26877
-0.000000       17.776764       0.927432        26880
-0.000000       13.798201       0.752604        26883
-0.000000       8.808613        0.752604        26886
-0.000000       0.639982        1.034668        26889
-0.000000       10.766092       1.034668        26892
-0.000000       16.904821       0.445843        26895
-0.000000       1.666170        1.558502        26898
-0.000000       -10.979050      1.558502        26901
[...]
[start to give wrong angle]
-63.827332      -55.537666      2.056338        61070
-64.664154      -45.493412      1.079513        61091
-63.179108      -56.125523      2.296158        61112
-64.335503      -45.969860      1.644291        61133
-63.493549      -55.467552      1.371975        61154
-64.448853      -45.504547      2.196720        61175
-63.668858      -54.957607      1.737229        61196
-64.651138      -44.757927      2.688058        61217
-63.793468      -54.527122      1.958851        61263
-64.480934      -45.149158      1.526006        61284
-63.927254      -53.981602      1.544853        61305
-64.703789      -44.238575      1.942603        61326
-63.879242      -53.901367      2.547676        61347
-64.262489      -45.579575      2.409893        61368
-63.828140      -53.842373      1.538354        61389
-64.043808      -46.200710      1.969249        61410
-63.806774      -53.689724      2.068687        61431
-64.063972      -45.942078      1.305034        61452
-64.021843      -52.690182      1.516257        61473
-64.294502      -44.844070      1.698884        61519
```

```
-65.281967      -46.305569      0.819546        61540
-64.285416      -44.677036      1.532505        61561
-65.201920      -46.542126      1.843165        61582
-64.377304      -43.942196      1.465563        61603
-65.903488      -41.367073      1.852914        61624
-64.222763      -44.822174      1.455165        61645
-65.240486      -35.436710      1.696934        61666
-63.959816      -45.156792      2.342952        61687
-64.874809      -36.300003      1.282287        61708
-64.070786      -44.632587      1.719031        61729
-65.198074      -35.077415      0.929382        61775
-63.960823      -44.621716      1.249791        61796
-65.261963      -34.687939      1.780123        61817
-63.459919      -44.796219      1.735279        61838
-64.947678      -34.976135      1.875661        61859
-63.055786      -44.614552      2.437840        61880
-64.916275      -34.910343      2.338402        61901
-63.116257      -44.662010      1.625443        61922
-64.662140      -35.071354      1.785973        61943
-63.172806      -44.486954      2.076486        61964
-64.432076      -35.392708      1.709282        61985
-62.762703      -44.721329      1.231593        62006
-64.219391      -35.592239      1.776224        62052
-62.828789      -44.548218      1.924405        62073
-64.278320      -35.365868      1.625443        62094
-63.307785      -43.624279      0.902735        62115
-64.465637      -34.729813      1.664438        62136
-63.927181      -41.855431      1.205597        62157
-64.646248      -33.980755      1.487661        62178
-63.967247      -41.629181      1.410321        62199
-64.677994      -33.784012      2.017993        62220
-64.214119      -40.723686      1.761926        62241
-63.420967      -39.025414      1.982898        62262
-63.957047      -41.926178      1.806770        62308
-64.250778      -35.551479      1.973149        62329
-63.976246      -41.803623      1.375875        62350
-64.302353      -35.329327      1.751527        62371
-64.050781      -41.489307      1.729430        62392
-66.016380      -34.703491      2.431990        62413
-64.035919      -41.670391      1.138005        62434
-65.695976      -37.804218      1.902308        62455
-63.920612      -42.573296      1.912057        62476
-65.840134      -36.391243      2.724453        62497
-63.820911      -43.656895      1.729430        62518
-65.881927      -33.026939      2.403394        62539
-63.829029      -43.696453      1.426568        62585
-65.960144      -33.780243      1.509758        62606
-63.823776      -43.569176      0.941080        62627
-65.357796      -33.245827      1.763875        62648
-63.792156      -43.535007      1.189349        62669
-65.594139      -32.827084      1.587098        62690
-63.625084      -43.254177      1.989397        62711
-65.663437      -32.982861      1.404471        62732
-63.565216      -42.874943      2.300057        62753
-65.357628      -32.590805      1.873711        62774
-63.540524      -42.841835      1.741778        62795
-64.399887      -34.697773      1.574750        62841
-63.873295      -41.689064      0.971626        62862
-62.497265      -42.863914      1.365476        62883
-64.133583      -39.566624      0.955378        62904
```

```
-62.143639      -48.472527      1.601396        62925
-64.067711      -39.538010      1.430468        62946
-62.519554      -49.138615      1.613745        62967
-64.131676      -39.391582      2.216868        62988
-62.110455      -48.129360      1.430468        63009
-64.043060      -39.202400      1.678736        63030
-62.778378      -48.571861      2.265612        63051
-64.069145      -39.010437      2.330603        63097
-62.812836      -48.427925      1.745678        63118
-63.889011      -39.331955      2.673760        63139
-62.227024      -48.837284      1.270588        63160
-63.801586      -39.346199      1.022320        63181
-61.914204      -48.388588      1.440867        63202
-63.790462      -39.357285      1.568900        63223
-61.826790      -45.290298      1.563051        63244
-63.713428      -39.949490      1.948452        63265
-62.682365      -39.563526      0.967727        63286
-63.739822      -39.656105      1.241992        63307
-62.005817      -43.484612      1.816519        63328
-63.487949      -41.379688      1.700834        63374
-64.510185      -41.904140      2.269511        63395
-63.411766      -41.702488      1.276438        63416
-64.915115      -38.966576      1.314783        63437
-63.440022      -41.426399      2.064787        63458
-62.433163      -50.178440      1.518207        63479
-63.466766      -41.302826      2.032291        63500
-62.835506      -49.194618      1.646240        63521
-63.492451      -41.172974      1.534455        63542
-63.306564      -47.631504      2.346851        63563
-63.482025      -41.177940      1.883460        63584
-63.594891      -46.452972      1.810670        63630
-63.605011      -40.644157      1.915956        63651
-63.859386      -45.275944      0.994373        63672
-63.749302      -40.001572      1.325181        63693
-62.792847      -48.720867      1.264089        63714
-63.810535      -39.797424      1.895809        63735
-62.299953      -49.255013      1.177000        63756
-63.893570      -39.663754      1.298535        63777
-62.430790      -49.169231      2.147977        63798
-63.861263      -39.685322      1.790522        63819
-62.535358      -49.024998      1.915956        63840
-63.700890      -40.031189      1.209496        64077
-62.495144      -49.032639      2.005645        64098
-63.755451      -39.921955      2.292258        64119
-61.757919      -47.159107      1.260190        64140
-63.805771      -39.597076      1.938703        64161
-61.916111      -44.807102      1.812619        64182
-63.811325      -39.951527      1.855514        64203
-65.441414      -36.484135      1.812619        64224
-63.745564      -40.642342      1.800271        64245
-65.594788      -35.449131      1.700834        64266
-63.542137      -43.111153      1.536404        64287
-65.028633      -41.030273      1.170501        64308
-63.476814      -43.665470      1.719031        64355
-65.555580      -33.780384      1.148404        64376
-63.462620      -43.040756      1.563051        64397
-65.596184      -34.115253      1.993296        64418
-63.496063      -43.308670      1.215345        64439
-65.417038      -37.697842      1.944553        64460
-63.490829      -43.511189      1.867212        64481
```

```
-65.324387      -38.749310      1.491560        64502
-63.684502      -42.060707      1.973149        64523
-65.356895      -38.472401      0.929382        64544
-64.108131      -39.274658      1.231593        64565
-65.727806      -35.705723      1.579299        64609
-64.019257      -39.657394      1.761926        64630
-65.186920      -30.866619      1.440867        64651
-63.925495      -39.703255      1.430468        64672
-64.983162      -39.855026      1.420719        64693
-63.921936      -39.381046      1.552652        64714
-64.554527      -41.833549      2.149926        64735
-64.038734      -38.489918      0.927432        64756
-62.843433      -47.291882      1.485711        64777
-64.130722      -38.082321      1.386274        64798
-66.199318      -29.844561      1.794422        64819
-64.057358      -38.223068      1.719031        64840
-63.086720      -46.694927      2.038141        64887
-63.979092      -38.219448      0.965777        64908
-63.505646      -45.292538      1.574750        64929
-63.708626      -38.953373      2.259762        64950
-64.144814      -42.310669      1.639741        64971
-63.739929      -38.540974      1.850964        64992
-63.939926      -42.965435      2.105082        65013
-63.761257      -38.213535      1.700834        65034
-62.807224      -46.513531      1.370026        65055
-63.971676      -37.492878      1.999795        65076
-63.829762      -33.307056      2.210369        65097
-64.119614      -36.681099      1.960800        65141
-65.723091      -26.685959      1.397972        65162
-64.068153      -36.636948      0.829944        65183
-66.108894      -28.834433      2.547676        65204
-64.023834      -37.075745      2.088834        65225
-65.847717      -26.710947      1.371975        65246
-63.940403      -36.958237      1.928305        65267
-65.639015      -26.936647      2.454738        65288
-63.768181      -36.902004      1.331031        65309
-65.529320      -27.007524      2.391695        65330
-63.759350      -36.825573      1.115908        65351
-65.337494      -27.311848      1.619594        65398
-64.094666      -36.144997      1.741778        65419
-65.330612      -27.269592      3.148849        65440
-64.155014      -35.919666      1.105509        65461
-65.392860      -27.017550      1.967300        65482
-63.965977      -36.243114      1.668338        65503
-65.196053      -27.480970      0.935231        65524
-64.051422      -35.953655      1.514307        65545
-65.291069      -27.113613      2.023842        65566
-64.124092      -35.695072      1.215345        65587
-65.559578      -26.260109      1.944553        65608
-64.214096      -35.446976      1.459064        65629
-66.298317      -26.869370      1.565001        65673
-64.145599      -37.185169      2.531428        65694
-66.274559      -27.334185      1.637792        65715
-64.116554      -37.499466      1.709282        65736
-65.978188      -26.757614      1.709282        65757
-64.049095      -37.473900      1.538354        65778
-65.553574      -27.516479      2.702356        65799
-64.039856      -37.434807      1.205597        65820
-65.706154      -27.102457      1.392123        65841
-64.282669      -37.015011      1.828867        65862
```

```
-65.020607      -29.277315      2.381297        65883
-63.737156      -38.428799      1.635842        65973
-64.875969      -29.634926      2.105082        65994
-63.645325      -38.574970      1.574750        66015
-64.810562      -29.767698      1.745678        66036
-63.579758      -38.668011      2.840138        66057
-64.497261      -30.665134      2.227267        66078
-63.648232      -38.421768      2.105082        66099
-64.162437      -31.763962      1.832767        66120
-63.789276      -37.868286      2.368948        66141
-63.961159      -32.475525      2.247414        66162
-63.702587      -38.139431      1.495460        66183
-63.412930      -34.617077      1.359627        66227
-63.576241      -38.664322      2.789445        66248
-65.065796      -29.103865      1.475312        66269
-63.631691      -38.581161      1.893859        66290
-65.664734      -30.976276      1.722931        66311
-63.674816      -38.209198      2.320205        66332
-63.981342      -42.271828      1.989397        66353
-63.624249      -38.460022      1.960800        66374
-63.266132      -35.240185      1.684586        66395
-63.558784      -38.745789      2.446289        66416
-62.081425      -40.928162      1.765825        66437
-63.586693      -38.591759      1.971199        66458
-62.841587      -46.421787      2.541826        66505
-63.774158      -37.996212      2.365049        66526
-63.078709      -45.757778      2.247414        66547
-63.556324      -38.640991      1.363526        66568
-63.356705      -44.687496      1.485711        66589
-63.466949      -38.845699      1.989397        66610
-62.946026      -45.927505      2.303957        66631
-63.479706      -38.651299      1.574750        66652
-61.666786      -47.483982      1.430468        66673
-63.430801      -38.659782      2.426141        66694
-61.870243      -47.652454      1.849015        66715
-63.640011      -38.530823      1.690435        66759
-61.628662      -46.634697      0.971626        66780
-63.458649      -37.908066      1.739829        66801
-61.552269      -46.482063      2.170074        66822
-63.319263      -37.753899      2.340352        66843
-61.958843      -46.473930      1.518207        66864
-63.186665      -37.923161      1.303084        66885
-62.421844      -45.442883      1.451265        66906
-63.105358      -38.050266      1.790522        66927
-62.439079      -45.283302      1.325181        66948
-63.158760      -37.790886      1.552652        66969
-63.093163      -43.073353      1.655989        66990
-63.205101      -37.545620      2.458637        67038
-63.313812      -42.129425      2.210369        67059
-63.127571      -37.809082      1.408371        67080
-64.771378      -33.895573      1.408371        67101
-63.140705      -37.704941      2.231166        67122
-65.011803      -31.930775      2.430041        67143
-63.058365      -38.461380      1.751527        67164
-64.861603      -33.298115      1.623493        67185
-62.809368      -41.152107      1.154903        67206
-64.653969      -35.678684      2.043990        67227
-62.981987      -39.633461      1.652090        67248
-64.349724      -37.705704      2.115481        67292
-62.823441      -40.472839      2.038141        67313
```

```
-64.316238      -37.759357      2.043990        67334
-62.574036      -42.056187      2.043990        67355
-64.256119      -38.021461      2.227267        67376
-62.380230      -43.461628      1.264089        67397
-64.331802      -36.860634      0.878038        67418
-62.280945      -45.049404      1.495460        67439
-64.304657      -36.644192      1.396022        67460
-62.256306      -45.002445      1.103559        67481
-64.277710      -36.836285      1.749577        67502
-62.237408      -45.517426      2.413793        67549
-64.259651      -37.111366      1.778174        67570
-62.278435      -46.564365      1.540304        67591
-64.189758      -36.884468      1.832767        67612
-62.150780      -45.052380      2.076486        67633
-64.118111      -35.902130      1.922455        67654
-62.087242      -43.750778      2.533378        67675
-64.092964      -35.126976      2.015393        67696
-62.481159      -38.594254      2.346851        67717
-64.084770      -35.063972      1.816519        67738
-62.521282      -38.224705      1.849015        67759
-63.961781      -35.802208      1.335580        67780
-61.981075      -42.539722      1.655989        67824
-63.965752      -35.623024      2.164225        67845
-62.005695      -42.057789      1.816519        67866
-64.019714      -34.571453      2.263662        67887
-62.294064      -38.841686      1.684586        67908
[...]
-41.490017      -13.153864      2.480734        249072
-40.390972      -19.109512      2.426141        249093
-41.480431      -13.153530      2.695857        249139
-40.392838      -18.800350      3.230089        249160
-41.438419      -13.490382      2.586671        249181
-40.579933      -17.487501      2.287709        249202
-41.366409      -13.944941      2.426141        249223
-40.627060      -17.208967      2.586671        249244
-41.319744      -14.238596      2.448238        249265
-40.677479      -16.921293      2.612668        249286
-41.422096      -13.642299      2.844688        249307
-40.869759      -15.818869      2.879133        249328
-41.281048      -14.467551      2.168124        249349
-40.908684      -15.614155      2.393645        249395
-41.288635      -14.432008      2.663361        249416
-40.549362      -17.643396      2.911629        249437
-41.357239      -14.020188      3.477707        249458
-40.725090      -16.598269      2.708205        249479
-41.261230      -14.572355      3.406866        249500
-40.962471      -15.291872      2.222717        249521
-41.204155      -14.872356      3.291181        249542
-40.676468      -16.811771      2.978571        249563
-41.161034      -15.095714      2.017993        249584
-40.324265      -18.826763      3.264534        249605
-41.130974      -15.253028      2.917479        249651
-40.483501      -17.848949      2.762798        249672
-41.165474      -15.020706      2.370898        249693
-40.193531      -19.629856      2.728353        249714
-41.205105      -14.716257      3.126752        249735
-40.125225      -20.163166      2.844688        249756
-41.208427      -14.631255      2.216868        249777
-40.266201      -20.155277      3.059811        249798
-41.130562      -14.942696      2.823891        249819
```

```
-40.829212      -17.401703      3.098156        249840
-41.111881      -15.003064      3.419215        249861
-40.848145      -17.259621      2.101182        249907
-41.134609      -14.847544      3.618089        249928
-40.372242      -19.588688      3.126752        249949
-41.177692      -14.618860      2.933727        249970
-40.180328      -20.287951      3.004568        249991
-41.181644      -14.581749      4.062633        250012
-40.618484      -18.385027      3.143000        250033
-41.111828      -14.893708      3.011067        250054
-40.985447      -16.393471      3.137151        250075
-41.093452      -14.952674      2.823891        250096
-40.640022      -18.201910      2.521679        250117
-41.152508      -14.622684      3.246337        250163
-41.148880      -15.423343      2.791395        250184
-41.095177      -14.900814      2.756949        250205
-40.912285      -16.716499      2.923328        250226
-41.092285      -14.884315      2.960373        250247
-40.726986      -17.680199      2.756949        250268
-41.157883      -14.528523      3.116353        250289
-40.521420      -18.679613      2.675709        250310
-41.202873      -14.289539      2.630865        250331
-40.919800      -16.625214      2.879133        250352
-41.166607      -14.449601      2.708205        250373
-40.515373      -18.665449      2.245464        250394
-41.022873      -15.112987      2.994819        250440
-41.049576      -14.059886      3.078008        250461
-41.634586      -10.499047      3.204092        250482
-41.366932      -12.578590      2.576272        250503
-41.229393      -14.076200      2.476835        250524
-40.650555      -16.228548      2.177873        250545
-41.026665      -15.155120      2.675709        250566
-41.551067      -12.870124      2.560024        250587
-40.943489      -15.588847      2.944125        250608
-42.037933      -9.624268       2.487233        250629
-40.931755      -15.538546      1.897758        250650
-41.905491      -10.543240      3.065660        250696
-40.985382      -15.133140      2.923328        250717
-41.113945      -15.162821      3.484206        250738
-41.711689      -10.687736      3.953447        250759
-40.552464      -17.879536      3.137151        250780
-40.918900      -16.463856      2.769298        250801
-40.283470      -18.612087      1.715132        250822
-39.894638      -20.434307      2.509331        250843
-40.773415      -16.006605      2.042040        250864
-39.737846      -21.921467      3.634337        250885
-40.817696      -16.332525      2.917479        250906
-41.532406      -11.993667      2.001745        250952
-40.514206      -16.909189      3.264534        250973
-40.103493      -20.037180      2.669860        250994
-40.079651      -21.095137      2.852487        251015
-40.279114      -19.055668      3.660984        251036
-40.881004      -14.953219      3.364622        251057
-40.317215      -17.084164      2.389746        251078
-41.134285      -13.687127      2.653612        251099
-40.129436      -19.316694      3.435463        251120
-40.406948      -18.685028      2.277960        251141
-40.395813      -17.782732      3.397118        251162
-41.344864      -12.887370      2.740701        251208
-41.054531      -15.268892      2.856386        251229
```

```
-40.069431      -21.620501      1.958851        251250
-40.390568      -18.812241      3.016916        251271
-40.583950      -16.777945      3.380220        251292
-40.840500      -14.563026      3.397118        251313
-41.155415      -13.708478      3.132601        251334
-40.683239      -17.028791      3.297030        251355
-40.688511      -16.124647      2.911629        251376
-39.968037      -18.992598      3.280782        251397
-40.831692      -14.208650      2.277960        251418
-40.854271      -14.979665      2.641264        251464
-40.779572      -16.229988      1.881510        251485
-41.126423      -13.595401      2.547676        251506
-40.762379      -14.560862      2.592520        251527
-41.046642      -13.981068      2.978571        251548
-41.061855      -14.668586      2.730303        251569
-41.104149      -15.105277      3.143000        251590
-41.129604      -14.245300      3.297030        251611
-40.880890      -14.805438      2.791395        251632
-40.606556      -15.270251      2.895381        251653
-40.374847      -17.342093      3.287282        251674
-40.859276      -15.591951      2.519729        251720
-40.906326      -14.503891      1.969249        251741
-40.724735      -14.486345      2.773197        251762
-40.753990      -15.268579      2.675709        251783
-40.610039      -16.904325      2.630865        251804
-41.390713      -12.143756      3.297030        251825
-40.377972      -17.799683      3.098156        251846
-41.105961      -13.367455      2.437840        251867
-41.195621      -13.665914      3.711678        251888
-40.818649      -14.882685      3.402967        251909
-40.823460      -15.706033      3.750023        251930
-40.695564      -15.492135      3.098156        251976
-40.603661      -14.971440      3.114404        251997
[Lift off]
-40.612572      -15.876323      2.602919        252018
-40.536381      -17.180061      2.927877        252039
-40.729988      -15.267352      3.159248        252060
-40.825485      -15.619137      2.850537        252081
-40.871933      -14.500647      2.117430        252102
-41.592163      -11.279814      3.529051        252123
-40.679237      -15.618119      2.515180        252144
-41.264523      -13.183560      3.412716        252165
-40.648205      -15.731871      2.944125        252186
-40.733433      -16.125038      3.165097        252275
-40.804550      -16.474766      3.309379        252296
-40.640553      -18.198345      2.907080        252317
-41.186008      -14.371091      2.570423        252338
-41.599857      -9.703144       3.657084        252359
-40.729561      -15.584471      3.297030        252380
-40.877354      -16.648674      3.657084        252401
-41.061096      -13.849143      4.101628        252422
-41.367218      -7.705678       4.847083        252443
-40.740761      -8.285188       4.946520        252464
-40.594868      -12.342175      4.818487        252485
-41.268410      -8.060084       4.410339        252506
-41.504673      -5.690023       4.818487        252552
-41.807243      -2.784917       4.956919        252573
-41.665020      -4.847302       4.426587        252594
-41.694916      -3.818420       4.692403        252615
-41.455570      -4.696536       5.524947        252636
```

```
-41.452610      -2.127842       5.693925        252657
-41.261555      -3.171693       5.194139        252678
-41.074337      -4.410383       6.278851        252699
-41.159328      9.798386        5.088852        252720
-40.587746      5.387310        5.671828        252741
-39.794312      -81.853439      6.360091        252762
-39.273239      -140.968369     5.909048        252808
-38.634518      -152.937180     6.971663        252829
-38.102951      -158.479401     6.867676        252851
-37.455070      -162.970657     7.203033        252872
-36.659653      -157.959808     7.286222        252893
-36.079979      -158.215271     6.746141        252914
-35.541359      -160.389633     6.562865        252935
-34.984619      -161.258286     7.318718        252956
-34.286140      -160.721863     7.164038        252977
-33.835873      -161.371887     7.598833        252998
-33.279686      -162.318115     6.843629        253019
-32.782063      -162.745865     7.686572        253256
-32.209686      -161.634308     7.377211        253277
-31.939339      -163.319153     6.969063        253298
-31.508900      -164.020050     7.657975        253319
-31.079071      -164.397659     6.965164        253340
-30.771544      -166.050354     7.613131        253361
-30.294674      -166.581543     7.521492        253382
-29.813656      -167.097458     7.769761        253403
-29.271341      -167.656403     7.273874        253424
-28.707644      -167.835907     7.239429        253445
-28.214661      -168.517456     7.227080        253466
-27.495268      -168.252594     6.885874        253512
-26.864088      -168.572479     7.001559        253533
-26.137312      -168.526550     6.679200        253554
-25.435444      -168.639709     6.943066        253575
-24.734951      -169.059662     8.027128        253596
-23.999735      -169.225449     6.721445        253617
-23.281914      -169.776550     6.573263        253638
-22.494656      -169.522858     5.874602        253659
-21.806545      -169.705566     6.666852        253680
-21.204796      -170.287842     7.084748        253701
-20.570427      -170.789703     6.209960        253722
-19.799725      -170.725555     6.885874        253766
-19.107189      -171.110901     6.660352        253787
-18.380867      -171.620300     5.977939        253808
-17.584749      -172.159760     5.344270        253829
-16.792282      -172.538559     6.374389        253850
-16.061441      -173.200027     5.807661        253871
-15.284568      -173.664047     4.975117        253892
-14.469365      -173.853912     5.293576        253913
-13.826052      -174.653824     5.090802        253934
-13.079703      -175.049255     5.318273        253955
-12.392745      -175.719131     5.685476        253976
-11.655773      -176.279175     4.371994        254023
-10.809903      -176.929581     4.359645        254044
-9.957039       -176.487167     4.850983        254065
-9.100431       -176.497604     4.215364        254086
-8.180058       -176.911972     4.112027        254107
-7.188787       -177.176727     4.424637        254128
-6.163297       -177.185852     3.995692        254149
-5.166008       -177.289215     3.230089        254170
-4.173365       -177.682526     3.594043        254191
-3.137889       -178.048035     3.477707        254212
```

```
-2.124807
```

Second Test

The second version of the code is provided in listing A.6.

Listing A.6

```
/**
 ******************************************************************************
 * @file    main.cpp
 * @author  Pierluca De Felice e Maria Mattiello
 * @version V1.2.3
 * @date    22-April-2023
 * @brief   Unmanned model plane stabilization using the X_NUCLEO_IKS01A3
 *          MEMS Inertial & Environmental Sensor Nucleo expansion board.
 ******************************************************************************
*/

//NOTE: when the MCU is connected to a computer, to print data on screen the "printf" function
//      should be uncommented. Remember to put them in comment again before preparing the MCU for the launch.

/* Includes */
#include "mbed.h"
#include "rtos.h"
#include "XNucleoIKS01A3.h"
#include "IMUfilter.h"
#include "SDBlockDevice.h"
#include "FATFileSystem.h"
#include <cstdio>

/* Gravity at Earth's surface in m/s/s */
#define g0 9.812865328f
/* Std air density */
#define density 1.225f
/* Conversion from rad to deg and from deg to rad */
#define todeg 57.2957795f
#define torad 0.01745329252f
/* Number of samples for ground level pressure */
#define SAMPLES 50


/* Defining SD device for storing data */
SDBlockDevice sd(D11, D12, D13, D10); //MOSI MISO SCK CS
FATFileSystem FSys("FSys");

/* Instantiate the expansion board */
static XNucleoIKS01A3 *mems_expansion_board = XNucleoIKS01A3::instance(D14, D15, D4, D5, A3, D6, A4);

/* Retrieve the composing elements of the expansion board */
static LPS22HHSensor *press_temp = mems_expansion_board->pt_sensor;
static LSM6DSOSensor *acc_gyro = mems_expansion_board->acc_gyro;

/* Defining the user led as output */
DigitalOut led(LED1);

/* Defining the user button as input */
InterruptIn button(BUTTON1);
```

```
51
52  /* Define the pwm pin that will send the signal to move the servo */
53  PwmOut tailL(D3);
54  PwmOut tailR(D5);
55
56  /* Filter for angles calculation */
57  IMUfilter imuFilter(0.1, 10);
58
59  /* Defining an int for countng the phase of the mission reached */
60  int phase = 0;
61
62  /* User button callback. When the button is pressed phase is increased by one */
63  void phase_add()
64  {
65      phase++;
66  }
67
68  /* Setting a timer for mesuring time during the filght */
69  Timer tempo;
70  int t = 0;
71  int intervallo = 0;
72
73  /* Simple main function */
74  int main()
75  {
76      /* Attach callback to User button press */
77      button.fall(&phase_add);
78
79      uint8_t id;
80      /* Declaring variables for taking angle mesurements */
81      int16_t Accel[3], Gyro[3];
82      float Accms2[3], Gyrads[3];
83      float fullscale, pitch, roll;
84
85      /* Declaring variables for calibrating the sensor and calculating the altitude during the
         flight */
86      float init_press = 0.0f;
87      float altitude = 0.0f;
88      float pressure = 0.0f;
89      float med_press = 0.0f;
90      float press_buffer = 0.0f;
91
92      /* Setting LSM6DSO */
93      acc_gyro->set_x_odr(208.0f);
94      acc_gyro->set_g_odr(208.0f);
95      acc_gyro->set_x_fs(16.0f);
96      acc_gyro->set_g_fs(1000.0f);
97
98      /* Enable all sensors */
99      press_temp->enable();
100     acc_gyro->enable_x();
101     acc_gyro->enable_g();
102
103     /* Setting LPS22HH */
104     press_temp->set_odr(200.0f);
105
106     /* Mounting the SD device */
107     FSys.mount(&sd);
108
109     /* Creating a file on the SD device and starting the program */
```

```
110     FILE *fd = fopen("/FSys/mbed1.txt", "w");
111
112     if(fd==NULL) {
113         /* Something went wrong */
114         for(int i=0; i<50; i++){
115         led=!led;
116         ThisThread::sleep_for(3000ms); // NOTE: If the led turns on and off for 3s, something
     has gone wrong with the sd. The program will run anyway but data will not be stored.
117         }
118     }
119     else {
120         fprintf(fd, "Welcome SD!!\r\n");
121
122         //printf("It works!\r\n");
123     }
124
125
126     //printf("\r\n--- Starting new run ---\r\n");
127
128     /* Cheking if the sensors work properly */
129     press_temp->read_id(&id);
130     //printf( "LPS22HH  pressure & temperature   = 0x%X\r\n", id);
131     fprintf(fd, "LPS22HH  pressure & temperature   = 0x%X\r\n", id);
132
133     acc_gyro->read_id(&id);
134     //printf( "LSM6DSO accelerometer & gyroscope = 0x%X\r\n", id);
135     fprintf(fd, "LSM6DSO accelerometer & gyroscope = 0x%X\r\n", id);
136
137     acc_gyro->get_x_fs(&fullscale);
138     //fprintf(fd, "%f \r\n", fullscale);
139     printf( "%f \r\n", fullscale);
140
141     acc_gyro->get_g_fs(&fullscale);
142     //fprintf(fd, "%f \r\n", fullscale);
143     printf( "%f \r\n", fullscale);
144
145
146     /* Getting multiple mesurements with the LPS22HH pressure sensor to set the ground level
     */
147     for( int i=0; i<50; i++){
148
149         press_temp->get_pressure(&pressure);
150         pressure = pressure*100;
151         press_buffer = press_buffer + pressure;
152         //printf(" %d ", i);
153         ThisThread::sleep_for(5ms);
154
155     }
156     init_press = press_buffer / 50;
157     press_buffer=0.0f;
158
159
160     /* Printing initial data on the SD Card */
161     //printf("Ground Level Pressure: %f \r\n", init_press);
162     fprintf(fd, "Ground Level Pressure: %f \r\n", init_press);
163     fprintf(fd, "Pitch (deg) \t\t Roll (deg)\t\t Alt (m)\t\t Time (ms)\r\n");
164
165     /* Setting the servo parameters */
166     tailL.period_ms(20);
167     tailL.pulsewidth_us(1500);
```

```
168     tailR.period_ms(20);
169     tailR.pulsewidth_us(2500);
170 
171     /* Starting the timer */
172     tempo.start();
173     tempo.reset();
174     while (1) {
175 
176         /* Getting values from the sensor */
177         acc_gyro->get_x_axes_raw(Accel);
178         acc_gyro->get_g_axes_raw(Gyro);
179         t = tempo.read_ms();
180 
181         /* Converting the values from the LSM6DSO into mesures to be used in the filter (rad/
    s and m/s/s) */
182         for(int i=0; i<3; i++){
183 
184             Accms2[i]=Accel[i];
185             Accms2[i]=(Accms2[i]*g0)/1000;
186             Gyrads[i]=Gyro[i];
187             Gyrads[i]=(Gyrads[i]*torad)/1000;
188             }
189 
190         /* NOTE: Gyro and accelerometer values must be converted to rad/s and m/s/s
191             before being passed trough the filter. */
192         imuFilter.updateFilter(Gyrads[0],
193                                Gyrads[1],
194                                Gyrads[2],
195                                Accms2[0],
196                                Accms2[1],
197                                Accms2[2]);
198         imuFilter.computeEuler();
199 
200         pitch = imuFilter.getPitch()*todeg; //to degrees
201         roll = imuFilter.getRoll()*todeg; //to degrees
202 
203         //printf("pitch = %f \r\n", pitch);
204 
205         /* Calculating the altitude from pressure data with the Hydrostatic Equation*/
206         press_temp->get_pressure(&med_press);
207         altitude = - ((med_press*100 - init_press) / (1.225f * 9.812865328f)) ;
208         fprintf(fd, "%f \t %f \t %f \t %d \r\n", pitch, roll, altitude, t);
209 
210 
211 
212         /* The variable "phase" indicates the mission phase. To manually advance the phase
    press the User Button.
213             To get from phase 0 to phase 1 is necessary to press the User Button once.
214             To get from phase 1 to phase 2 the aircraft should change its orientation from
    vertical to the specified angle.
215             To get from phase 2 to phase 3 is necessary to press the User Button once.
216            NOTE: The Launch Phase (case 1) should only start when the aircraft is mounted on
    the launchpad! Remember to press
217                   the User Button for the first time only after placing the aircraft in the
    desired position. */
218 
219         switch(phase){
220 
221             case 1: // Launch Phase: the aircraft will wait to reach the angle specified in
    the if() to start the Gliding Phase
```

```
222
223         if(pitch<-35.00){
224             //printf("Still vertical\r\n");
225         } else {
226             //printf("Starting glider phase\r\n");
227             phase++;
228             /* Setting the stable tailplane position. */
229             // tailL.pulsewidth_us(1800);
230             // tailR.pulsewidth_us(2200);
231         }
232
233         break;
234
235         case 2: // Gliding Phase: the aircraft changes the opening angle of the tailplane
    in order to keep a stable
236                 //                position at different pitch angles.
237
238         ThisThread::sleep_for(19ms);
239
240         if (pitch>-30.00 && pitch <-27.00)
241                 {//printf("stable \r\n");
242                  tailL.pulsewidth_us(2050);
243                  tailR.pulsewidth_us(1950);
244                  }
245         else if (pitch>-27.00 && pitch<-24.00)
246                 {//printf("too low... \r\n  correcting... \r\n");
247                  tailL.pulsewidth_us(2150);
248                  tailR.pulsewidth_us(1850);
249                  }
250         else if (pitch>-24.00 && pitch<=-20.00)
251                 {//printf("too low... \r\n  correcting... \r\n");
252                  tailL.pulsewidth_us(2200);
253                  tailR.pulsewidth_us(1800);
254                  }
255         else if (pitch>-20.00 && pitch<=-16.00)
256                 {//printf("too low... \r\n  correcting... \r\n");
257                  tailL.pulsewidth_us(2300);
258                  tailR.pulsewidth_us(1700);
259                  }
260
261         if (pitch<=2.00 && pitch >= -1.00)
262                 {//printf("stable \r\n");
263                  tailL.pulsewidth_us(1800);
264                  tailR.pulsewidth_us(2200);
265                  }
266         else if (pitch>1.00 && pitch<=3.00)
267                 {//printf("too low... \r\n  correcting... \r\n");
268                  tailL.pulsewidth_us(1600);
269                  tailR.pulsewidth_us(2400);
270                  }
271         else if (pitch>3.00 && pitch<=4.00)
272                 {//printf("too low... \r\n  correcting... \r\n");
273                  tailL.pulsewidth_us(1550);
274                  tailR.pulsewidth_us(2450);
275                  }
276         else if (pitch>4.00 && pitch<=5.00)
277                 {//printf("too low... \r\n  correcting... \r\n");
278                  tailL.pulsewidth_us(1500);
279                  tailR.pulsewidth_us(2500);
280                  }
```

```
281
282             else if (pitch>=-2.00 && pitch<-1.00)
283                     {//printf("too high... \r\n  correcting... \r\n");
284                      tailL.pulsewidth_us(1800);
285                      tailR.pulsewidth_us(2200);
286                      }
287             else if (pitch>=-3.00 && pitch<-2.00)
288                     {//printf("too high... \r\n  correcting... \r\n");
289                      tailL.pulsewidth_us(1900);
290                      tailR.pulsewidth_us(2100);
291                      }
292             else if (pitch>=-4.00 && pitch<-3.00)
293                     {//printf("too high... \r\n  correcting... \r\n");
294                      tailL.pulsewidth_us(2000);
295                      tailR.pulsewidth_us(2000);
296                      }
297             else if (pitch>=-5.00 && pitch<-4.00)
298                     {//printf("too high... \r\n  correcting... \r\n");
299                      tailL.pulsewidth_us(1600);
300                      tailR.pulsewidth_us(2400);
301                      }
302
303             else if (pitch<-5.00)
304                     {//printf("too high... \r\n  correcting... \r\n");
305                      tailL.pulsewidth_us(2200);
306                      tailR.pulsewidth_us(1800);
307                      }
308
309             break;
310
311             case 3: //Closing Phase: the aircraft has landed and the user operates the User
    Button to close the file on the SD Card
312
313             fclose(fd);
314             //printf("closing...\r\n");
315
316             while(phase>=3){
317                 led=!led;
318                 ThisThread::sleep_for(500ms);
319                 //printf("closed\r\n");
320             }
321
322             break;
323
324             }
325
326
327
328     }
329 }
```

Bibliography

Other sources

[2] S. O. Madgwick, "*An efficient orientation filter for inertial and inertial/magnetic sensor arrays*," Ph.D. dissertation, University of Bristol, 2010. [Online]. Available: https://x-io.co.uk/downloads/madgwick_internal_report.pdf.

[3] R. A. Ibrahim, *Liquid sloshing dynamics: theory and applications*. Cambridge University Press, 2005.

[4] F. T. Dodge *et al.*, *The new" dynamic behavior of liquids in moving containers"*. Southwest Research Inst. San Antonio, TX, 2000.

[5] Z. Gu, Y. Lei, W. Qian, Z. Xiang, F. Hao, and Y. Wang, "An experimental study on the mechanical properties of a high damping rubber bearing with low shape factor," *Applied Sciences*, vol. 11, no. 21, p. 10 059, 2021.

[6] M.-A. Xue, Z. Jiang, Y.-A. Hu, and X. Yuan, "Numerical study of porous material layer effects on mitigating sloshing in a membrane lng tank," *Ocean Engineering*, vol. 218, p. 108 240, 2020, ISSN: 0029-8018. DOI: https://doi.org/10.1016/j.oceaneng.2020.108240. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0029801820311628.

[7] V. G. Carcano, S. Paul, L. Lagunegrand, S. Pernon, Y. Guerin, and J.-P. Mazel, "Design, modelling and prototyping of a single-material composite tank for cryogenic propellant storage,"

[8] T. Ida, R. Komatsu, H. Hirano, and T. Morikawa, "Study of sloshing control method with damping device of actual floating roof tank," *Journal of Japan Society of Civil Engineers, Ser. A2 (Applied Mechanics (AM))*, vol. 67, no. 2, I_935–I_943, 2011.

[9] M. Sivỳ, M. Musil, O. Chlebo, and R. Havelka, "Sloshing effects in tanks containing liquid," in *MATEC web of conferences*, EDP Sciences, vol. 107, 2017, p. 00 069.

[10] D. S. Purnomo, A. R. A. Besari, and Z. Darojah, "Control of liquid sloshing container using active force control method," in *IOP Conference Series: Materials Science and Engineering*, IOP Publishing, vol. 190, 2017, p. 012 007.

[11] M. Konopka, F. De Rose, H. Strauch, C. Jetzschmann, N. Darkow, and J. Gerstmann, "Active slosh control and damping-simulation and experiment," *Acta Astronautica*, vol. 158, pp. 89–102, 2019.

[12] E. Demirel and M. M. Aral, "Liquid sloshing damping in an accelerated tank using a novel slot-baffle design," *Water*, vol. 10, no. 11, p. 1565, 2018.

[13] P Nouraeidanesh, M. Kabiri, and M. Goudarzi, "An innovative roof shape in liquid storage tanks to reduce dynamic sloshing effects," *Journal of Applied Fluid Mechanics*, vol. 11, no. 1, pp. 127–136, 2018.

[14] X.-l. Zheng, X.-s. Li, Y.-y. Ren, Y.-n. Wang, and J. Ma, "Effects of transverse baffle design on reducing liquid sloshing in partially filled tank vehicles," *Mathematical Problems in Engineering*, vol. 2013, 2013.

[15] H. Snyder, "Sloshing in microgravity," *Cryogenics*, vol. 39, no. 12, pp. 1047–1055, 1999, ISSN: 0011-2275. DOI: https://doi.org/10.1016/S0011-2275(99)00120-4. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0011227599001204.

[16] W. Yang, T. Zhang, C Li, S. Li, and X Xu, "Numerical simulation of pitching sloshing under microgravity," *Journal of Applied Fluid Mechanics*, vol. 12, no. 5, pp. 1527–1537, 2019.

[17] J. Walker, *Fundamentals of Physics*, 10th ed. Wiley, 2007, p. 214. [Online]. Available: https://elearn.daffodilvarsity.edu.bd/pluginfile.php/1479845/mod_label/intro/fundamentals-of-physics-textbook.pdf.

Updated on: May 31, 2023